Republic of the Philippines
Tarlac State University
COLLEGE OF COMPUTER STUDIES
Tarlac City, Tarlac
Tel. No. (045) 6068173

Case Study: Simulation of CPU Scheduling Algorithm

A Case Study Presented to
The Faculty of the Bachelor of Science in Computer Science
College of Computer Studies

In Partial Fulfillment
of the Requirements for the Course
Operating System

Prepared by:

Tan, Gian Vincent P.

BSCS - 3B

Submitted to:

Mrs. Joanne Cura

Due Date: May 7, 2026

I. Introduction

Overview

Neovise (2026) defines CPU scheduling as the method that an operating system uses to determine which process in the ready queue is to be executed next. It is important because there can be more than one process that needs to be processed by the processor and the operating system must decide how to schedule them in a way that the system is efficient, responsive and fair. Rather than processes being executed randomly, CPU scheduling makes sure that system resources are properly utilised and that processes are processed in an orderly way (Goyal, 2024).

According to GeeksforGeeks (2020), there are a few metrics used to measure the performance of CPU scheduling algorithms. CPU Utilization is the amount of time the CPU is being used as opposed to being idle. Throughput is the number of processes executed per unit of time. Waiting Time is the time spent in the ready queue before being serviced by the CPU. Turnaround Time is the time from the arrival of a process until it is completed. Response Time is the time from the arrival of a process until it gets its first allocation of CPU. These factors are used to evaluate the performance of the scheduling algorithms.

CPU scheduling algorithms are either preemptive or non-preemptive. Preemptive scheduling allows the operating system to preempt a process and give the CPU to another process if necessary. With non-preemptive scheduling, a running process will only be interrupted when it voluntarily relinquishes control of the CPU. Preemptive scheduling tends to be more responsive, but non-preemptive scheduling is easier to implement (Waseem, 2025).

Purpose

This study presents a practical implementation of fundamental CPU scheduling algorithms in operating systems. It goes beyond theoretical discussion by demonstrating how different scheduling policies behave when applied to processes with varying arrival times, burst times, and priority levels. By developing a working simulator, the study provides a clearer view of how CPU

allocation decisions influence execution order, waiting time, turnaround time, and overall system performance. It also serves as an academic and analytical tool for examining the differences among scheduling strategies in a more concrete and measurable way.

Objectives

The program was developed with the following objectives:

- To implement six CPU scheduling algorithms, including First-Come, First-Served (FCFS), Shortest Job First (SJF), Shortest Remaining Time (SRT), Round Robin (RR), Priority Scheduling, and Priority Scheduling with Round Robin.
- To receive user input of process details such as arrival time, burst time, priority and time quantum (if needed).
- To execute the processes according to the chosen CPU scheduling algorithm.
- To produce a Gantt chart showing the order and time of CPU allocation for each process.
- To calculate and show the waiting time and turnaround time for every process.
- To compute the average waiting time and average turnaround time for evaluation.
- To demonstrate the effects and performance of preemptive and non-preemptive scheduling through test cases.

II. Body

A. First Come First Serve (FCFS)

First-Come, First-Served is the most basic scheduling algorithm in the simulator. It schedules processes in the order they are added to the ready queue: the first process in the queue is the first to be executed and the process continues to run until it completes before the next process is selected.

FCFS' scheduling decision-making is simple. The process simulator first orders the processes by arrival time, then selects the earliest process for execution. If there is no process to execute, the clock advances to the time of the next process's arrival. Once a process is chosen, it is allowed to run to completion and the next process in the queue is then run.

FCFS is a **non-preemptive** scheduling algorithm as the process that is running is not interrupted once it has started execution. The process will retain the CPU until its burst time is finished or it voluntarily releases the CPU (GeeksforGeeks, 2020).

Advantages

According to Team (2021), one major advantage of FCFS is its simplicity. It is easy to code, easy to understand, and requires minimal scheduling overhead. The algorithm only needs to maintain the arrival order of processes in the ready queue.

Another advantage is fairness based on arrival time. Every process is served according to who came first, which prevents disputes in execution order.

FCFS also has fewer context switches compared to preemptive algorithms since a running process is not interrupted until completion.

Limitations

Despite its simplicity, FCFS has several weaknesses. One of the most common problems is the convoy effect. This happens when a long process enters the CPU first while several short processes wait behind it. As a result, the short processes experience unnecessary delays.

FCFS can also lead to poor average waiting time, especially when long burst-time processes arrive before shorter ones.

Another limitation is that FCFS is not ideal for interactive systems because users may experience slow response time while waiting for long processes to finish (TheKnowledgeAcademy, 2024).

Test Cases

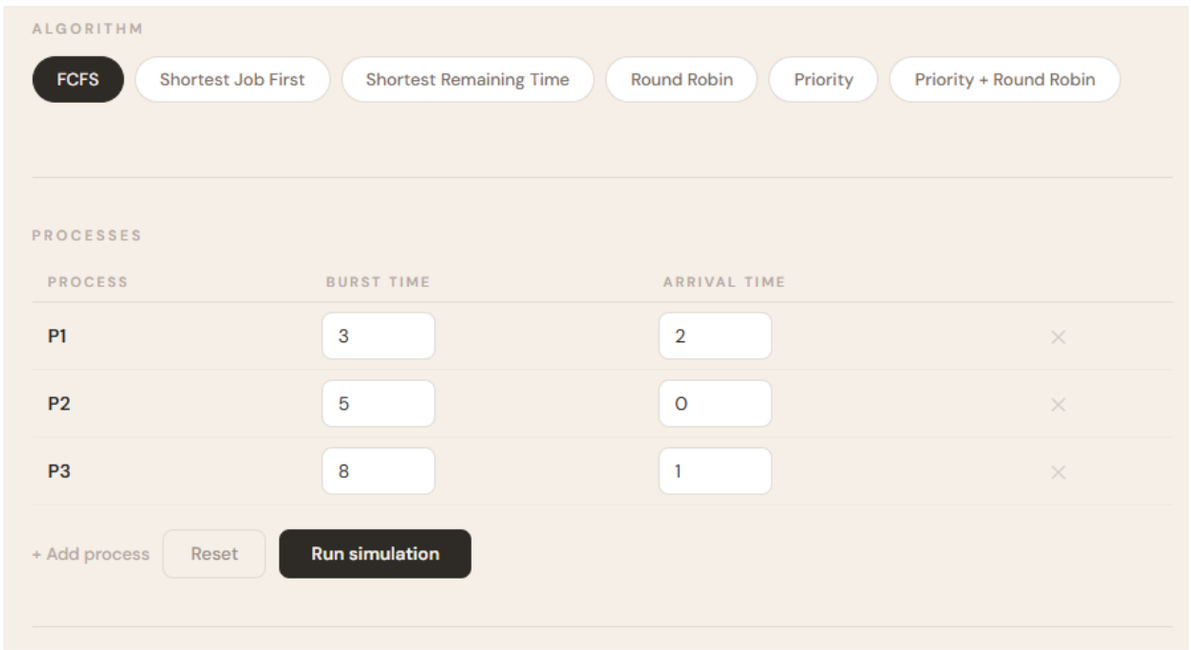

Figure 1. ***FCFS Test Case 1 (Input)***

Figure 2. ***FCFS Test Case 1 (Output)***

The output shows that P2 is executed first simply because it arrived first, even though P1 has a shorter burst time. This demonstrates that FCFS does not consider job length when making its scheduling decisions.

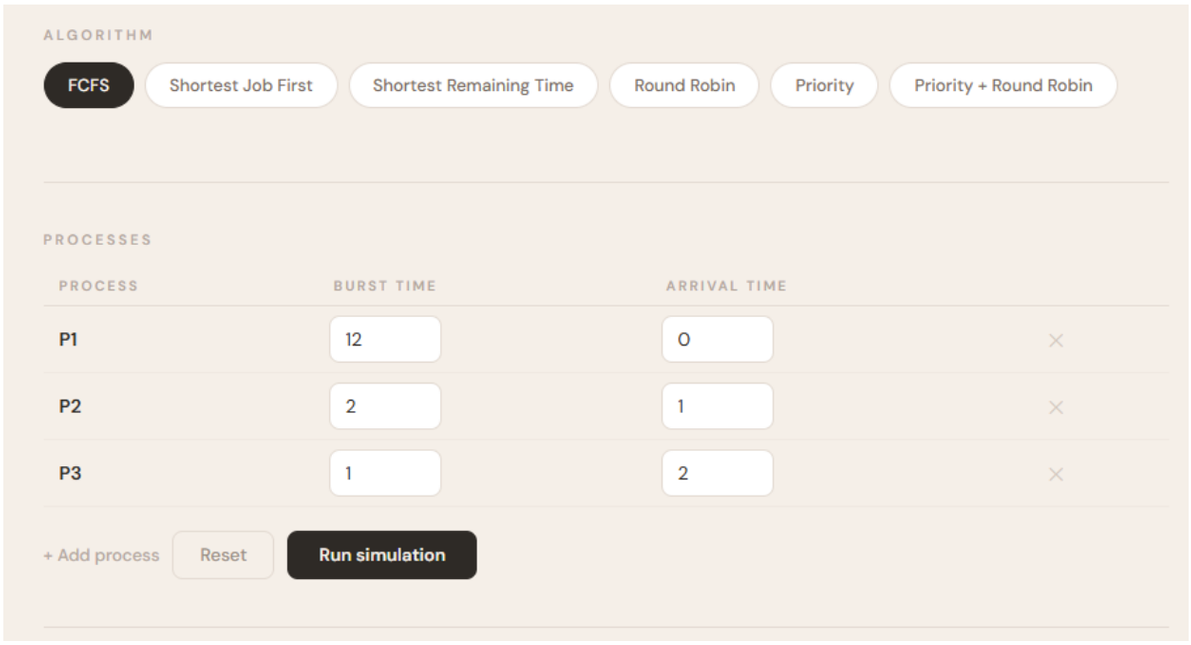

Figure 3. ***FCFS Test Case 2 (Input)***

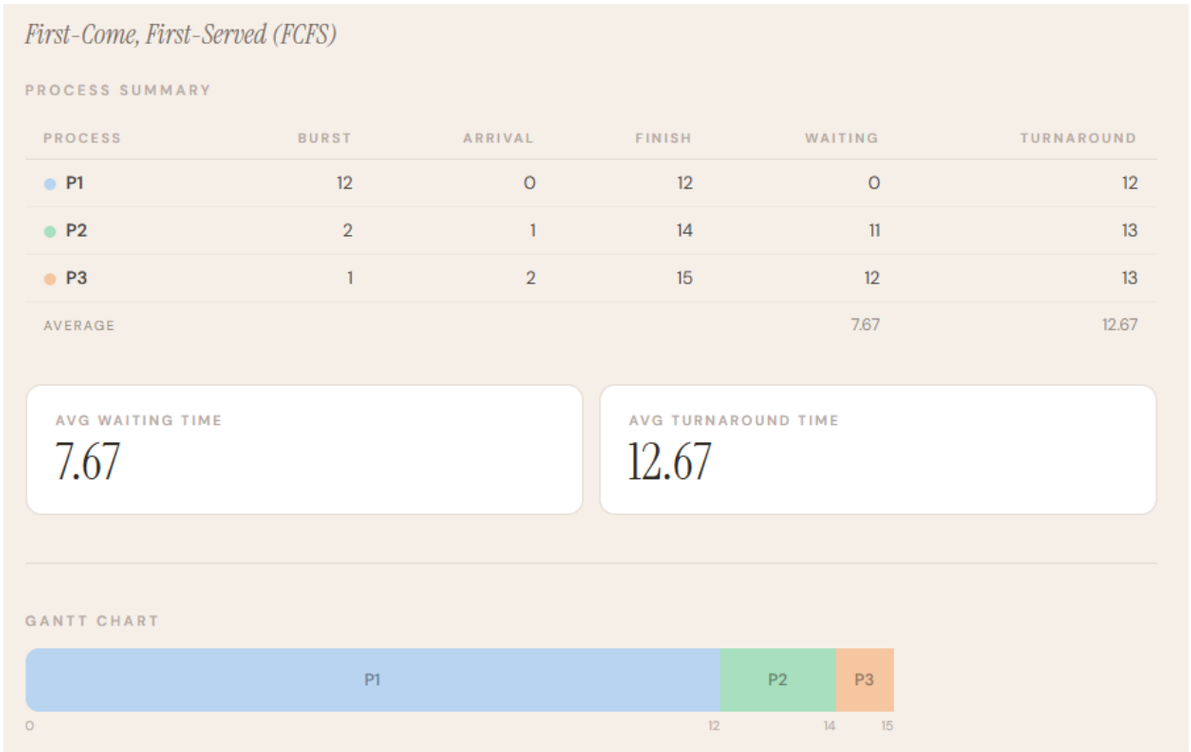

Figure 4. ***FCFS Test Case 2 (Output)***

Compared with Test Case 1, this case shows the convoy effect more clearly because the short jobs P2 and P3 must wait until the long-running P1 is completed. As a result, the average waiting time becomes much larger.

B. Shortest Job First (SJF)

As described in DIRA (2015), in Shortest Job First (SJF), the process with the shortest burst time from among the processes that have already arrived in the ready queue is chosen. In the simulator, when the CPU is free, the scheduler examines all processes, their burst times, and selects the one with the smallest burst time to run next.

The decision-making of SJF can be broken down into steps. First, the scheduler checks if there is a process that has arrived; second, if it has not arrived, the clock jumps to the next arrival; third, the process with the minimum burst time is selected among all ready processes; and fourth,

the selected process is run to completion, and a new process is chosen. In the case of a tie in burst time, the process that arrived first is chosen.

SJF is **non-preemptive** because once a process is chosen, it will not be interrupted by the arrival of a shorter process. Once a process has been chosen, it runs until it is completed.

Advantages

One of the biggest advantages of SJF is that it usually gives the lowest average waiting time among non-preemptive scheduling algorithms. It also improves turnaround time because shorter jobs complete earlier. SJF is efficient for batch systems where processes can be estimated before execution.

Limitations

The main disadvantage of SJF is the need to know or predict burst time in advance. In real operating systems, exact burst time is often unknown.

Another limitation is starvation. Long processes may keep waiting if short processes continue to arrive.

SJF may also be difficult to implement in dynamic systems where processes constantly enter and leave the ready queue.

Test Cases

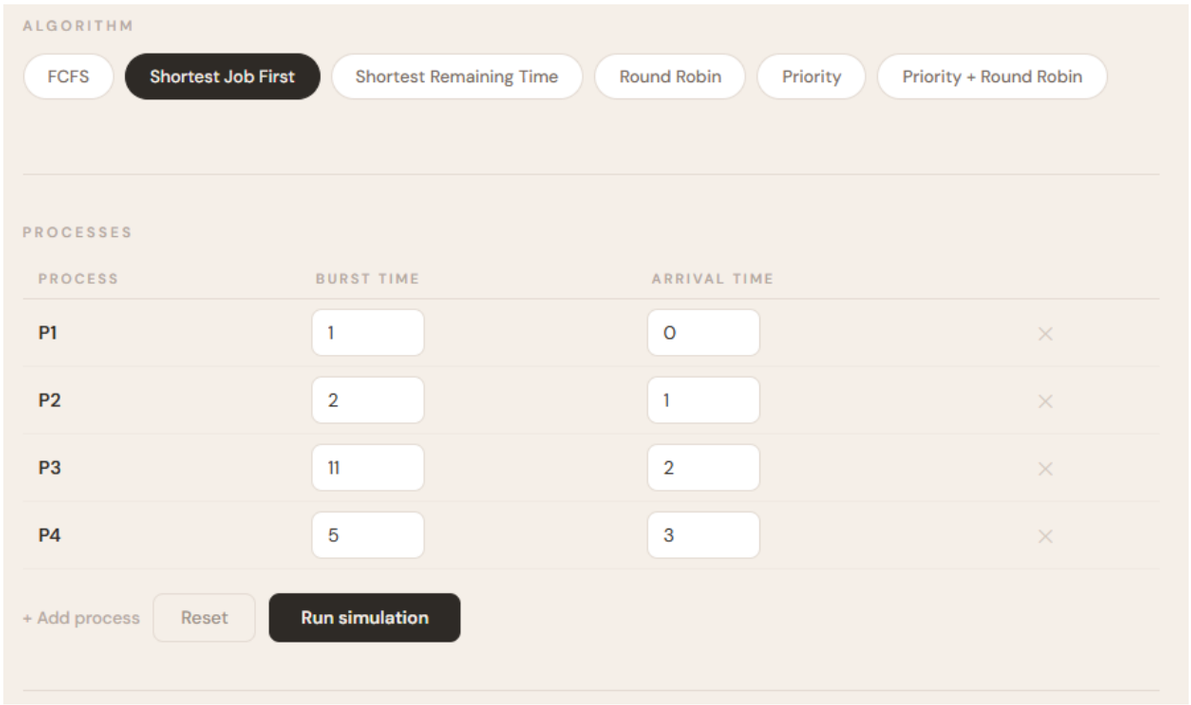

Figure 5. ***SJF Test Case 1 (Input)***

Figure 6. ***SJF Test Case 1 (Output)***

This output shows that after P1 finishes, the scheduler compares the waiting processes and chooses the shortest one first. That is why P2 runs before P4 and P3, clearly demonstrating how SJF prioritizes smaller burst times among arrived processes.

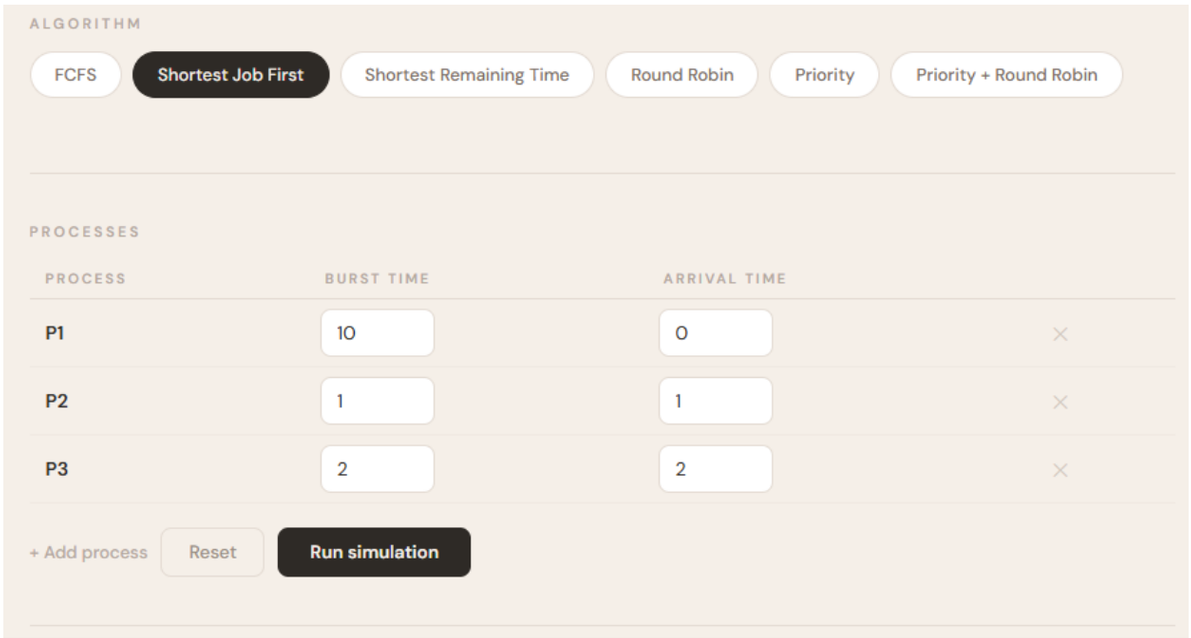

Figure 7. ***SJF Test Case 2 (Input)***

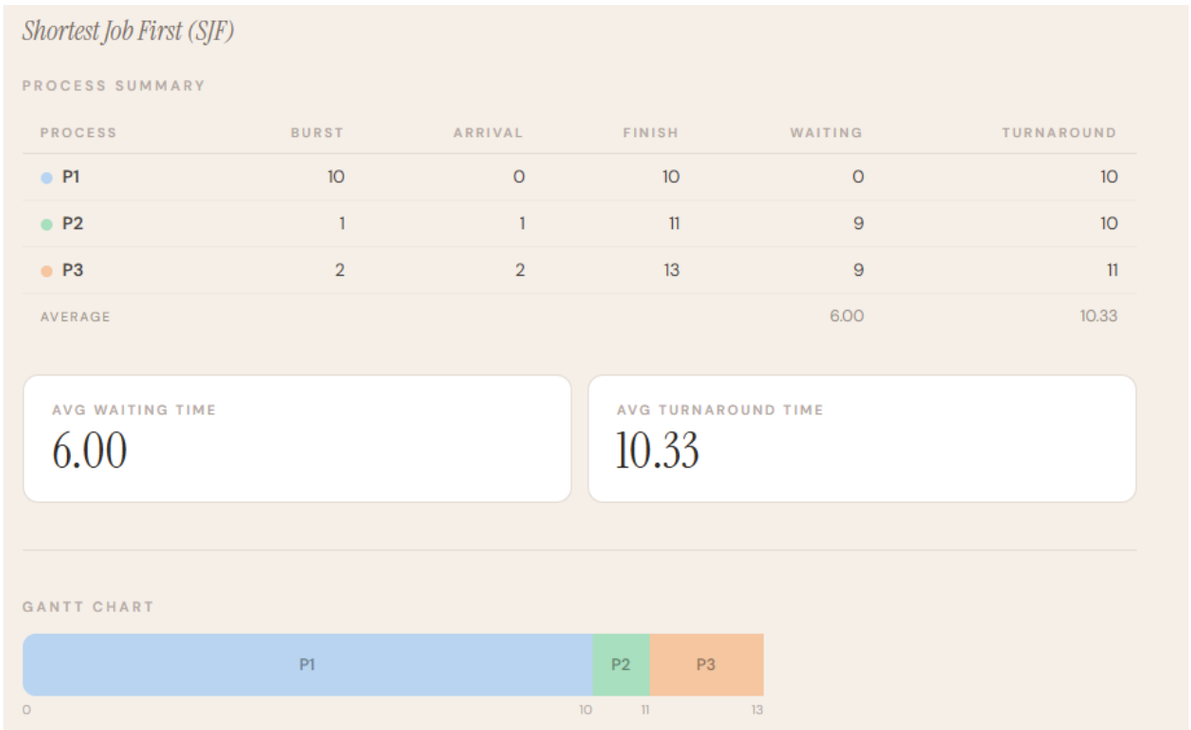

Figure 8. ***SJF Test Case 2 (Output)***

Compared with Test Case 1, this case highlights the non-preemptive nature of SJF more clearly. Even though shorter jobs arrive after P1 starts, they cannot interrupt it, so they still wait until P1 has fully completed.

C. Shortest Remaining Time (SRT)

Shortest Remaining Time is the preemptive version of SJF and is implemented in the simulator by checking, at every time step, which arrived process has the smallest remaining burst time. If a newly arrived process has a smaller remaining time than the currently running process, the running process is interrupted and the CPU is reassigned.

The decision-making process of SRT is more dynamic than that of FCFS or SJF. First, the scheduler checks all arrived but unfinished processes at each unit of time; second, it selects the process with the smallest remaining burst time; third, if that choice differs from the currently running process, a preemption occurs; and fourth, the process continues until either another shorter process arrives or it completes execution. In the simulator, consecutive blocks of the same process are merged later to keep the Gantt chart readable.

SRT is **preemptive** because the CPU can be taken away from the current process whenever a more favorable process, meaning one with shorter remaining time, becomes available. This is the defining behavior that distinguishes it from regular SJF (www.naukri.com, 2024).

Advantages

One of the biggest advantages of Shortest Remaining Time (SRT) is that it usually produces a lower average waiting time compared to many other scheduling algorithms. Since the process with the shortest remaining execution time is always selected, shorter tasks are completed quickly instead of waiting behind longer processes. This also helps improve turnaround time because more short jobs finish earlier.

Another advantage of SRT is its fast response time. If a new short process arrives while another process is running, the scheduler can immediately switch the CPU to the shorter task. Because of this, SRT is suitable for interactive systems where users expect quick responses and faster execution of small tasks.

SRT is also more adaptive than non-preemptive algorithms because it continuously checks the current workload and makes scheduling decisions based on the shortest remaining time. This allows better CPU utilization and can increase throughput when many short processes are present in the system (GeeksforGeeks, 2019).

Limitations

One major limitation of SRT is the high number of context switches. Since the running process may be interrupted whenever a shorter process arrives, the CPU spends additional time saving and loading process states. If this happens too often, system performance may decrease because of scheduling overhead.

Another disadvantage is the possibility of starvation for long processes. If short jobs continue arriving, longer processes may keep getting delayed and may wait a long time before finishing. This creates fairness issues because short processes are always favored over long ones.

SRT is also more difficult to implement compared to simpler algorithms like FCFS or SJF. The operating system must continuously monitor process arrivals, track remaining burst times, and make frequent scheduling decisions. It also depends on accurate burst time estimation, which may not always be possible in real systems (GeeksforGeeks, 2019).

Test Cases

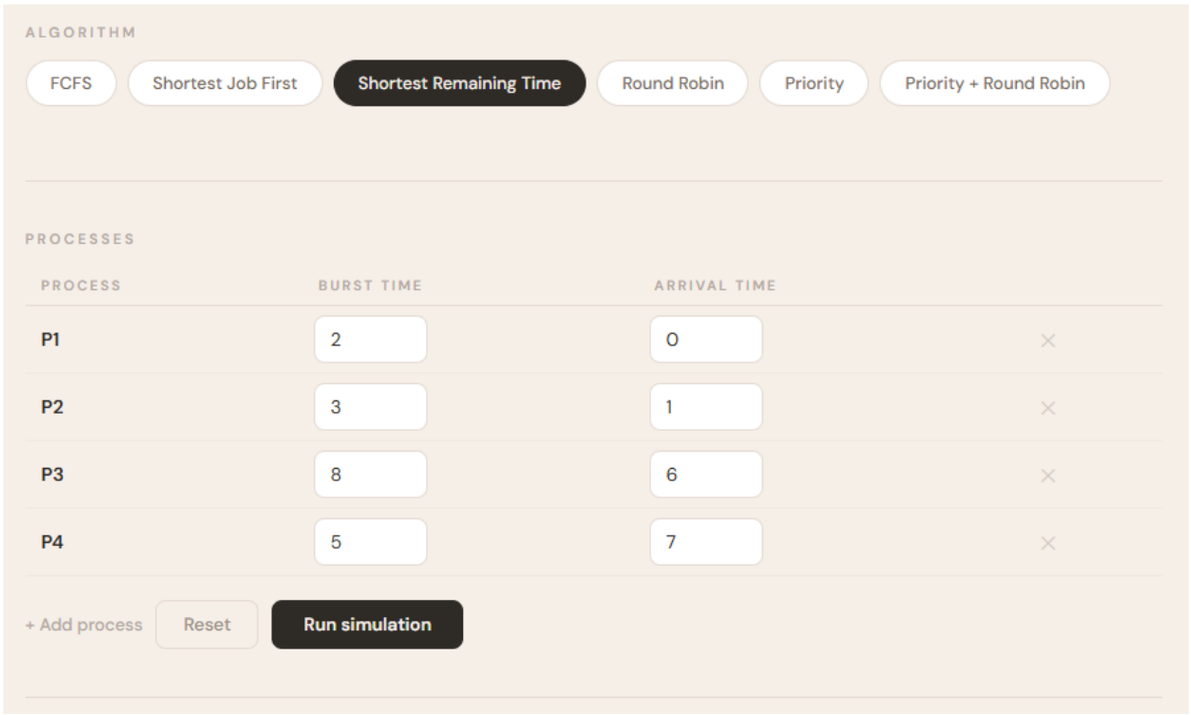

Figure 9. ***SRT Test Case 1 (Input)***

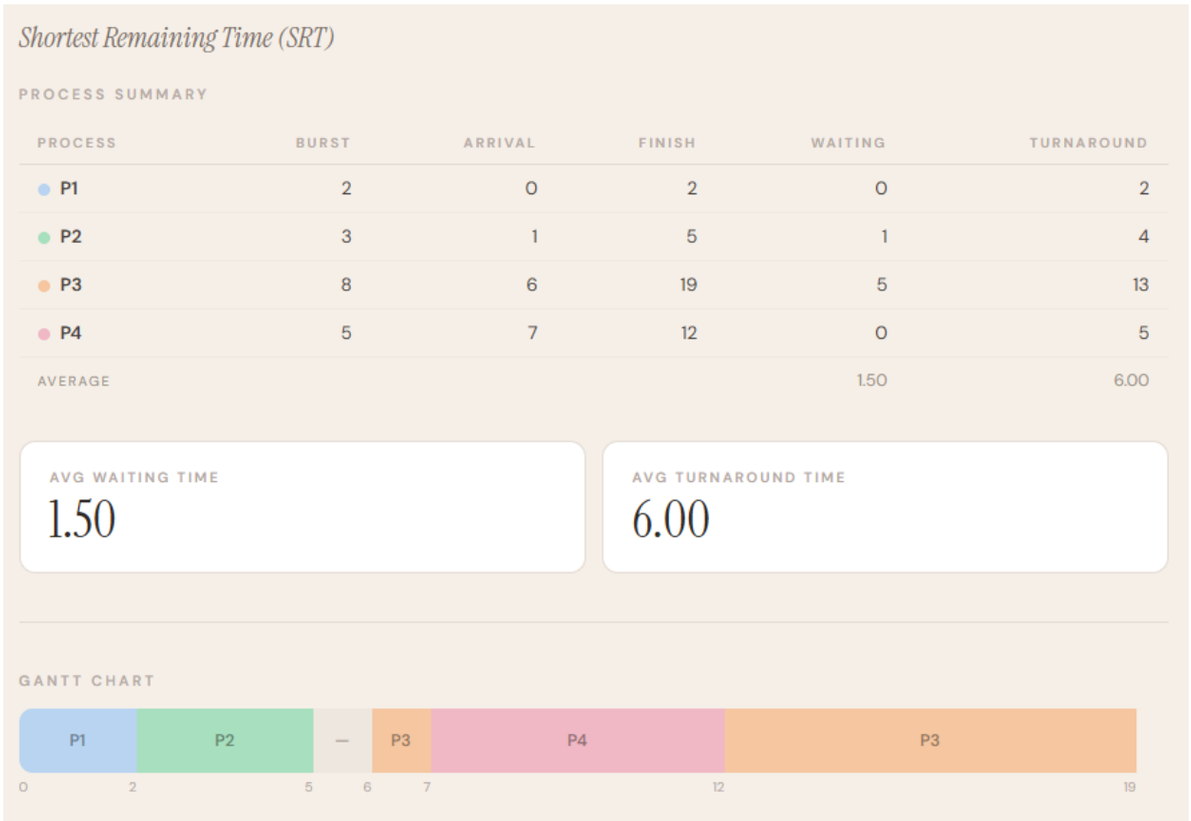

Figure 10. ***SRT Test Case 1 (Output)***

Figure 10 shows that P3 begins at time 6 but is interrupted at time 7 when P4 arrives with a shorter remaining time. This clearly demonstrates preemption and shows how SRT always favors the process that can finish sooner based on the remaining burst.

It also shows there is an idle time that started at 5 and ended at 6. This happens because a process finishes before the next one arrives.

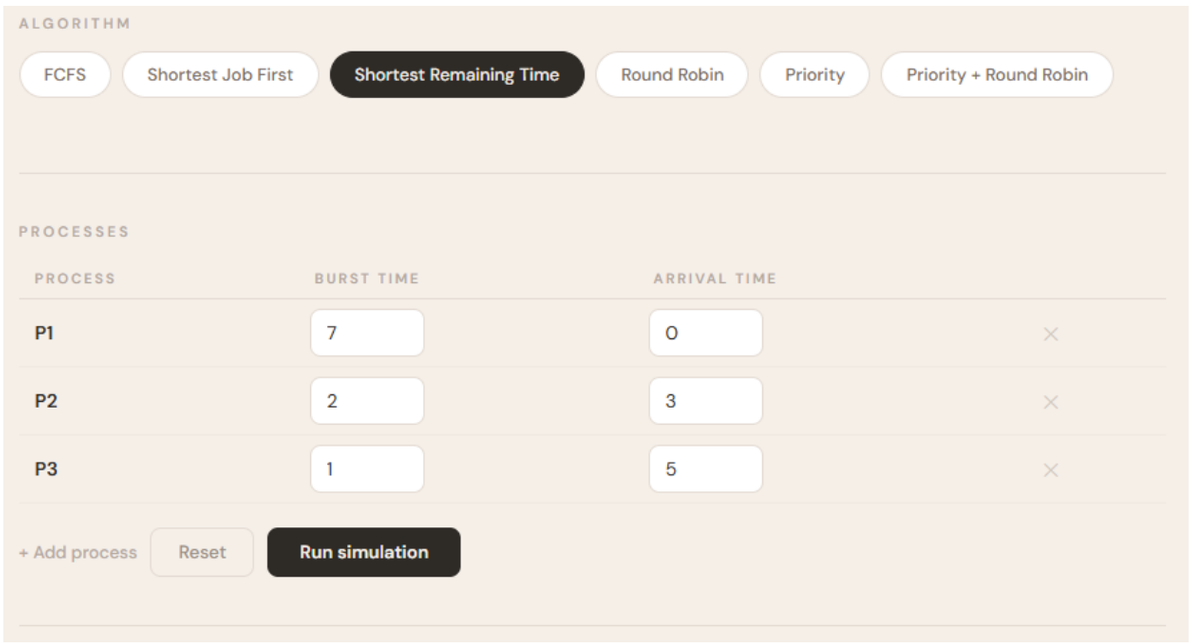

Figure 11. ***SRT Test Case 2 (Input)***

Figure 12. ***SRT Test Case 2 (Output)***

Compared with Test Case 1, this case shows an even stronger advantage for short late-arriving jobs because both P2 and P3 finish quickly with zero waiting time. The output reflects how SRT can greatly improve average waiting time when short processes appear after a long process has already started.

D. Round Robin (RR)

Round Robin is a scheduling algorithm that aims to fairly share the CPU among all processes in the ready queue by allocating a time quantum to each process. In the simulator, processes are organized in a circular queue, and each process is allowed to run for up to the given quantum time, and then returned to the end of the queue if it has more burst time to complete.

Round Robin's decision-making process is cyclical. First, all new processes are added to the ready queue; second, the process at the head of the queue is chosen; third, it is run either until the quantum expires or it completes, whichever happens first; fourth, new processes are added to the queue; fifth, new processes are added to the queue; and fifth, remaining processes are added to the end of the queue. This process repeats until all processes are finished.

Round Robin is **preemptive** since it can interrupt the running process at the end of its quantum. It does not hold the CPU until it is completed unless it completes within its time quantum (TutorialsPoint, 2025).

Advantages

A benefit of using Round Robin (RR) is its fairness. All processes get an equal amount of CPU time (time quantum). All processes are guaranteed to get a share of the CPU in a round-robin manner. It is therefore suitable for multi-tasking operating systems.

It also has a low response time. Because all processes are run in rapid succession, users do not have to wait for their processes to start. That's why Round Robin is popular in time sharing and interactive systems.

Round Robin also avoids starvation as all processes waiting for the CPU will eventually be served. If the time quantum is set correctly, it can also be both fair and efficient.

Limitations

A drawback of Round Robin is that its behaviour is dependent on the time quantum. If the time quantum is too small, the number of context switches increases, which introduces additional overhead and causes the CPU to be less efficient.

If the quantum is too big, Round Robin algorithm becomes FCFS. This slows down responsiveness as processes have to wait longer to get their next time slice.

Round Robin also has a longer average waiting time and turnaround time than SJF and SRT algorithms. Because the processes are executed in turns, short processes may have to wait for long ones (TutorialsPoint, 2025).

Test Cases

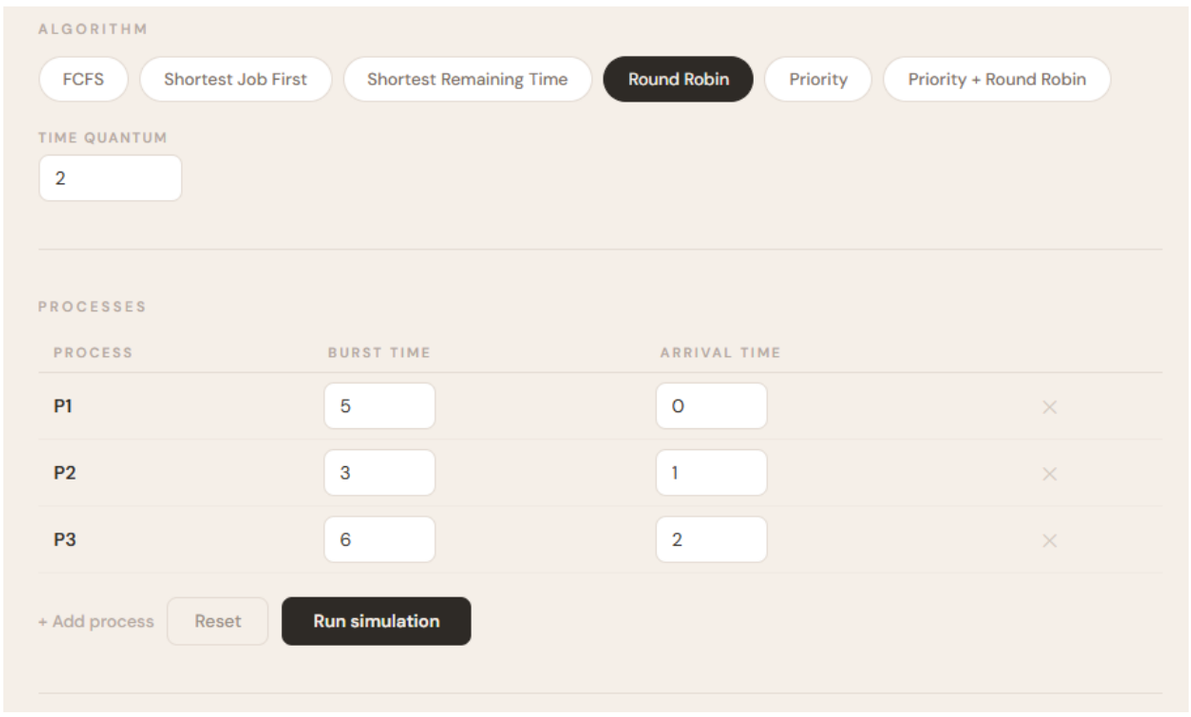

Figure 13. ***RR Test Case 1 (Input)***

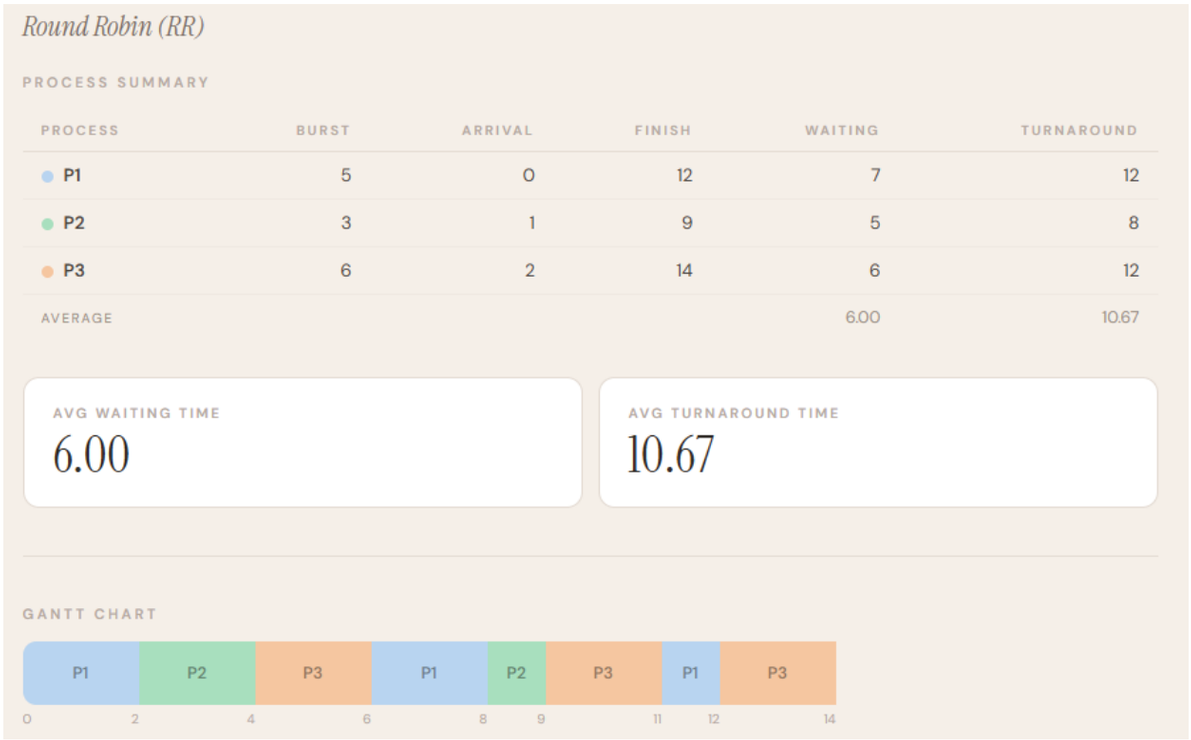

Figure 14. ***RR Test Case 1 (Output)***

The output clearly shows a cyclic execution order in which the processes alternate turns instead of running to completion one by one. This demonstrates the fairness of Round Robin and shows how the time quantum shapes the Gantt chart.

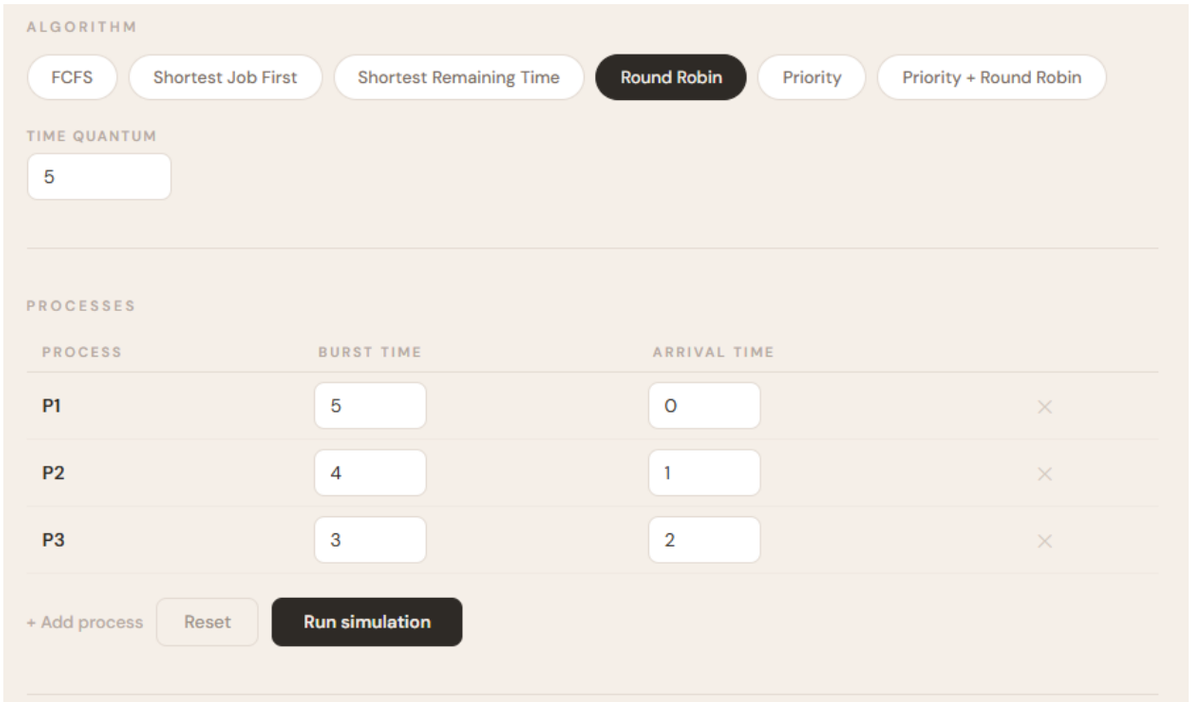

Figure 15. ***RR Test Case 2 (Input)***

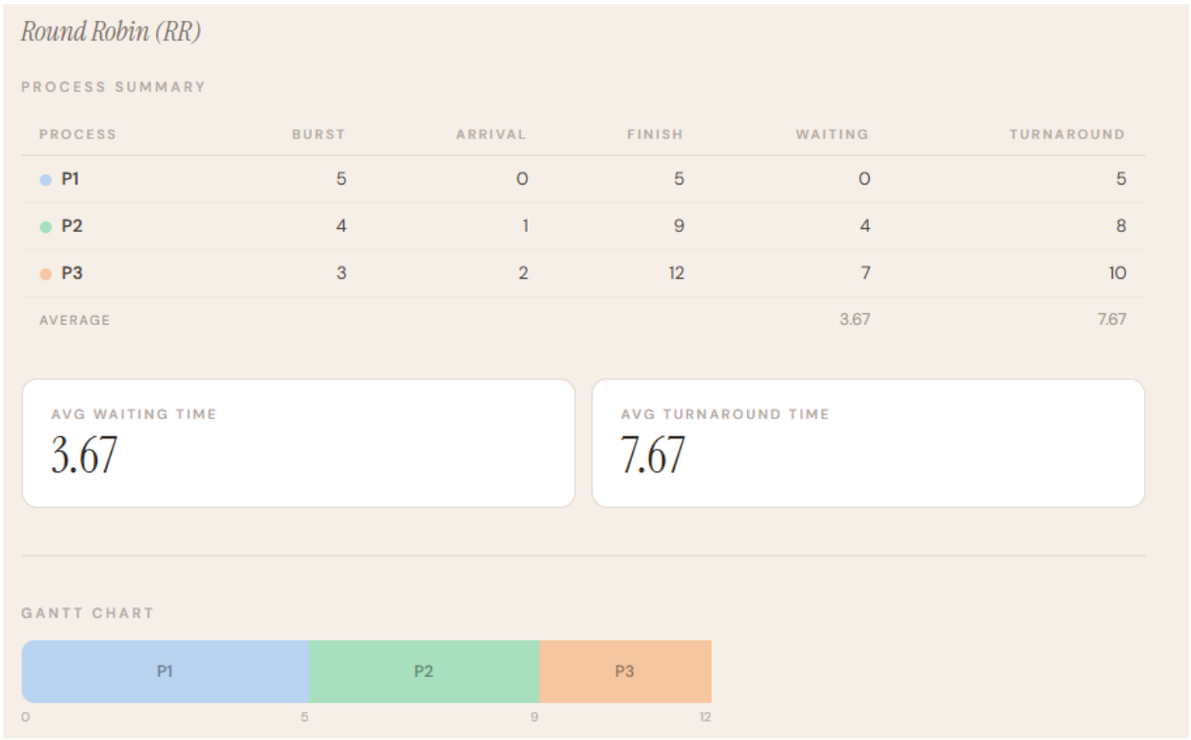

Figure 16. ***RR Test Case 2 (Output)***

Compared with Test Case 1, the larger quantum reduces the number of interruptions and makes the schedule look more like FCFS. This difference shows how the choice of time quantum significantly affects the behavior and performance of the Round Robin algorithm.

E. Priority Scheduling (Non-Preemptive)

The Priority Scheduling algorithm chooses the process with the highest priority from the processes that have arrived in the ready queue. In this simulator, the lower the number, the higher the priority, and if two processes have the same priority, the one that arrived first will be selected.

Both arrival and priority are considered in the decision-making. First, the scheduler finds the set of available processes that have arrived and not yet been completed; second, if there is no process, the time advances to the next arrival; third, the processes are sorted by priority; and fourth, the process with highest priority is run to completion. The running process is not interrupted (DataFlair, 2021).

This version of Priority Scheduling is **non-preemptive** because the running process is not interrupted by new processes, even if they have a higher priority. The next decision is made when the CPU becomes idle.

Advantages

A key benefit of Non-preemptive Priority Scheduling is that high priority processes are given preference over low priority processes. This is beneficial in situations where there are critical tasks that need to be performed before other less critical tasks.

Another benefit is that once a process is allocated the CPU, it will run until it completes. This makes the number of context switches lower, and increases the efficiency of the CPU compared with preemptive scheduling.

It is also easy to implement as the scheduler simply selects the process with the highest priority from the ready queue when the CPU is free. It works well for batch processing systems with pre-established process priorities.

Limitations

A major disadvantage of Non-preemptive Priority Scheduling is starvation. It can result in starvation of low-priority processes if there are many high-priority processes in the system.

Another problem is that once a process is running, it will not be interrupted even if a process with a higher priority enters the system. This might impact the responsiveness of the system.

It may also become unfair if priorities are not assigned properly. Wrong priority values can lead to the unfairness of some processes and unnecessary delays for others (DataFlair, 2021).

Test Cases

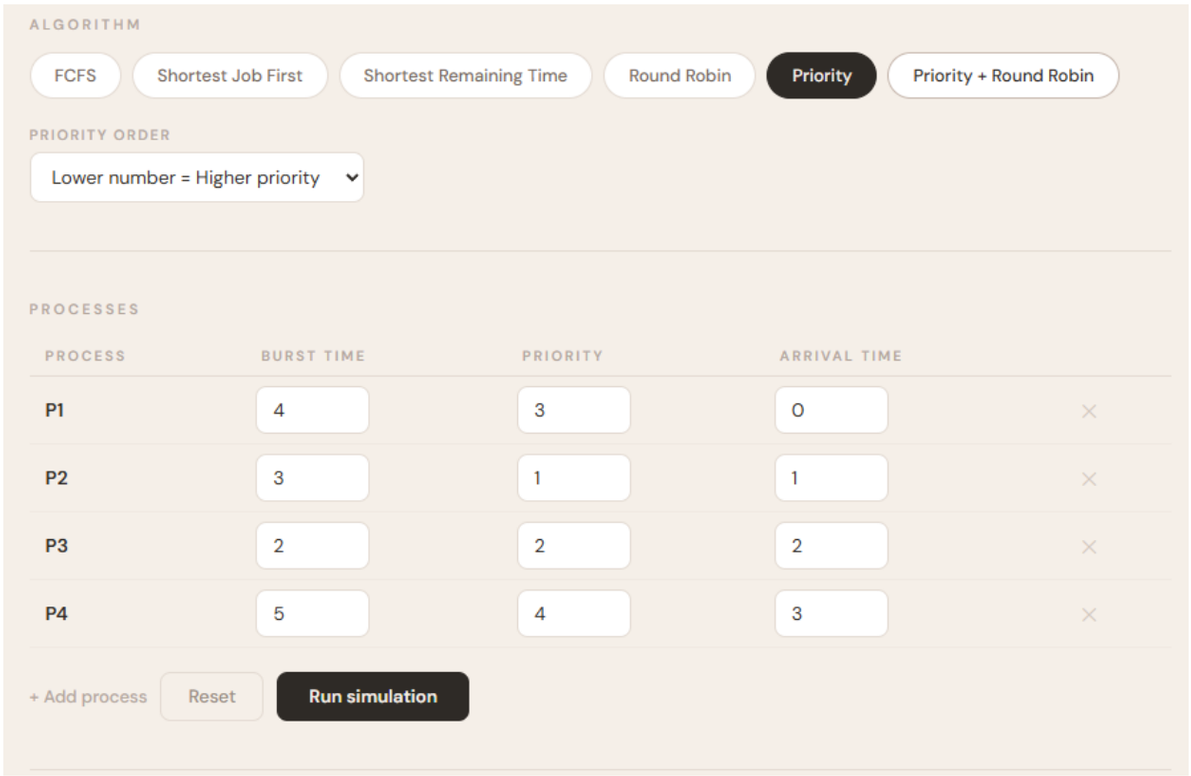

Figure 17. ***Priority Scheduling Test Case 1 (Input)***

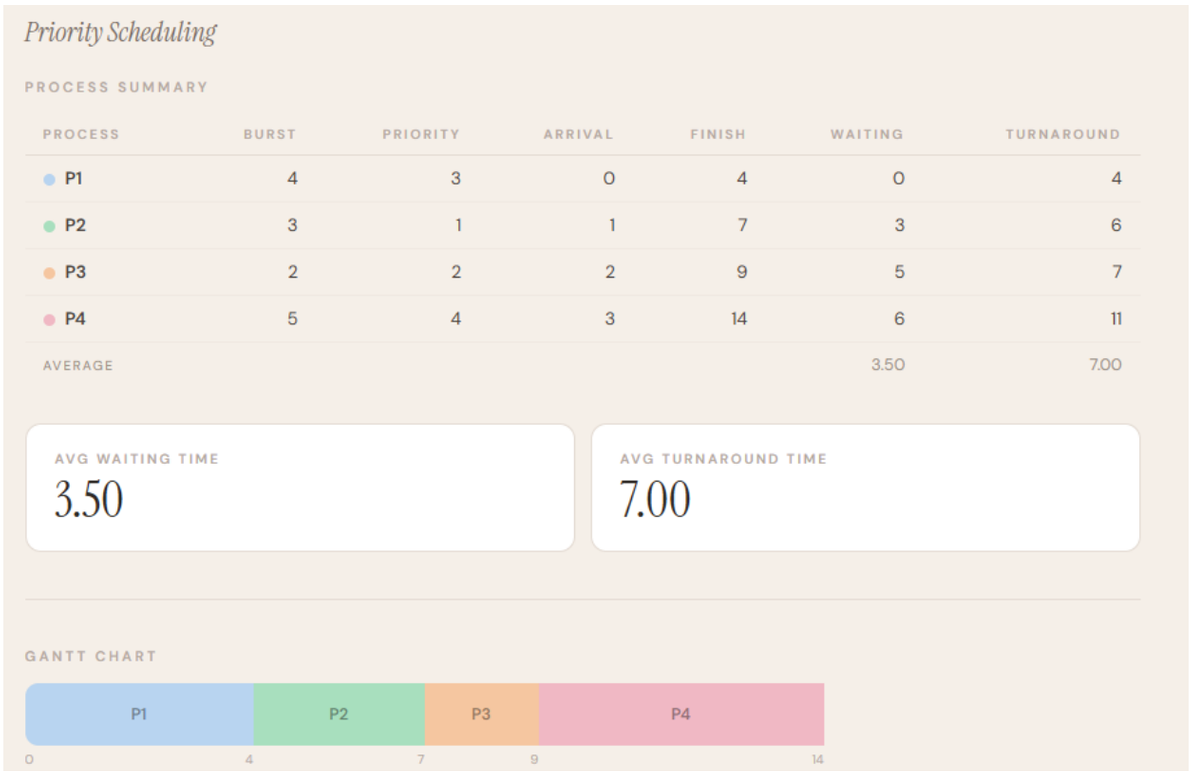

Figure 18. ***Priority Scheduling Test Case 1 (Output)***

This result shows that P1 executes first only because it is the only process available at time 0. After that, the scheduler chooses the next processes according to priority order, demonstrating that priority affects selection only among processes that have already arrived.

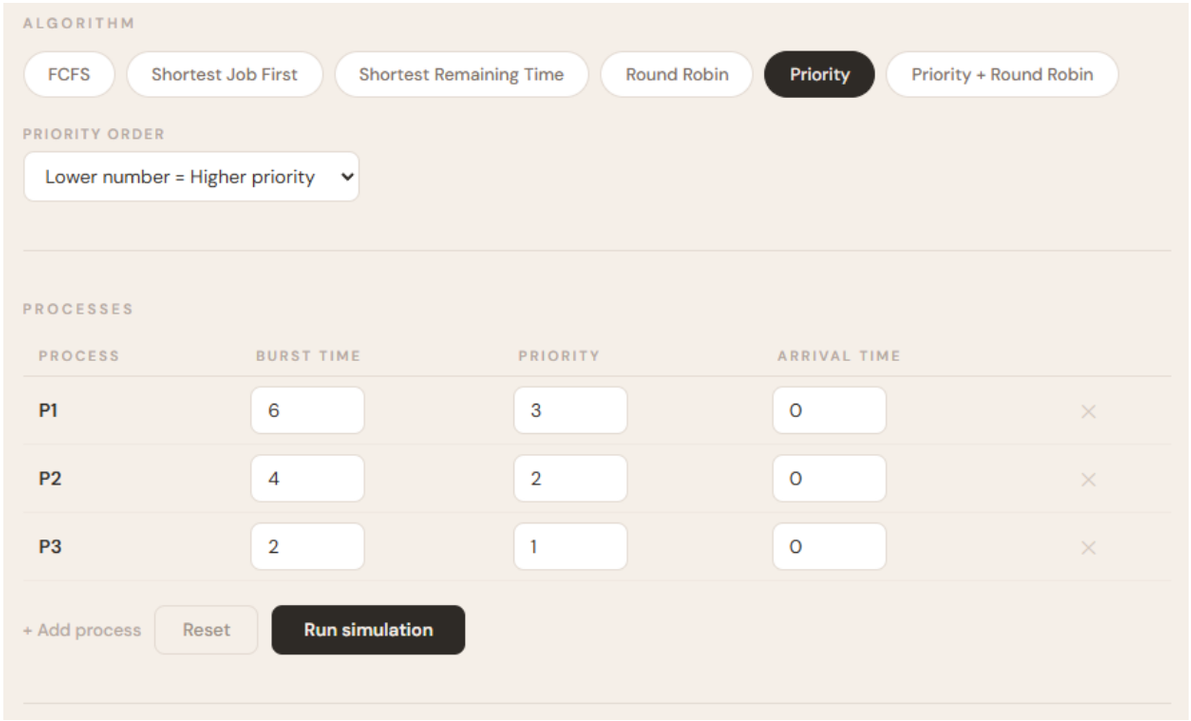

Figure 19. ***Priority Scheduling Test Case 2 (Input)***

Figure 20. ***Priority Scheduling Test Case 2 (Output)***

Compared with Test Case 1, this case removes arrival-time differences because all processes enter at the same moment. As a result, the output reflects pure priority ordering, making the role of priority more obvious than in the first case.

F. Priority Scheduling with Round Robin

Priority Scheduling with Round Robin is a hybrid algorithm that combines priority-based process selection with time-sharing among processes of the same level. In the simulator, processes are first ordered by priority, but within a given level, the CPU is shared using a time quantum in a Round Robin manner.

Its decision-making process has several stages. First, the scheduler gathers all processes that have already arrived; second, the ready queue is arranged according to priority; third, the highest-priority available process is selected; fourth, that process runs for at most one time quantum; and fifth, if it does not finish, it is placed back into the queue while newly arrived processes are added and the queue is sorted again by priority. This means the algorithm balances urgency with repeated CPU sharing.

Priority Scheduling with Round Robin is preemptive because a process can lose the CPU at the end of its quantum and return later for another turn. The algorithm therefore does not require a process to finish in one uninterrupted run (Kaushik, 2025).

Advantages

One advantage of this algorithm is that it combines the urgency control of Priority Scheduling with the fairness of Round Robin. Another advantage is that processes with the same priority level are given more balanced CPU access instead of being forced into strict one-pass execution order (Kaushik, 2025).

Limitations

One major limitation of Priority Scheduling with Round Robin is starvation of lower-priority processes. If high-priority processes continue arriving, lower-priority groups may wait for a long time before receiving CPU time.

Another disadvantage is the added complexity of implementation. The system must manage both priority queues and Round Robin scheduling, making it more difficult than using only one scheduling method.

Its performance also depends on the selected time quantum. If the quantum is too small, many context switches occur, while if it is too large, responsiveness becomes lower. Poor priority assignment may also reduce fairness in the system (Kaushik, 2025).

Test Cases

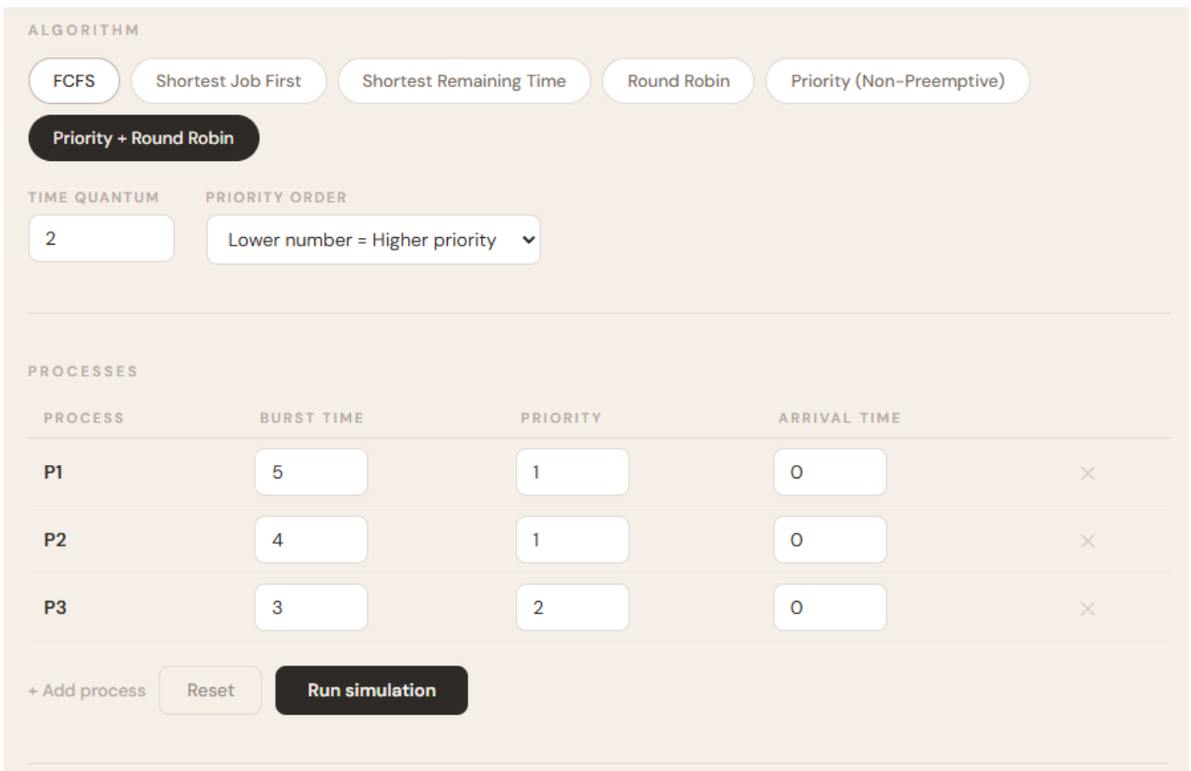

Figure 21. ***Priority with RR Test Case 1 (Input)***

Figure 22. ***Priority with RR Test Case 1 (Output)***

This output shows that P1 and P2 alternate because they belong to the same higher-priority group and therefore share the CPU using Round Robin. P3 waits until the higher-priority processes finish, which shows that priority still dominates the schedule even though time-sharing exists within a level.

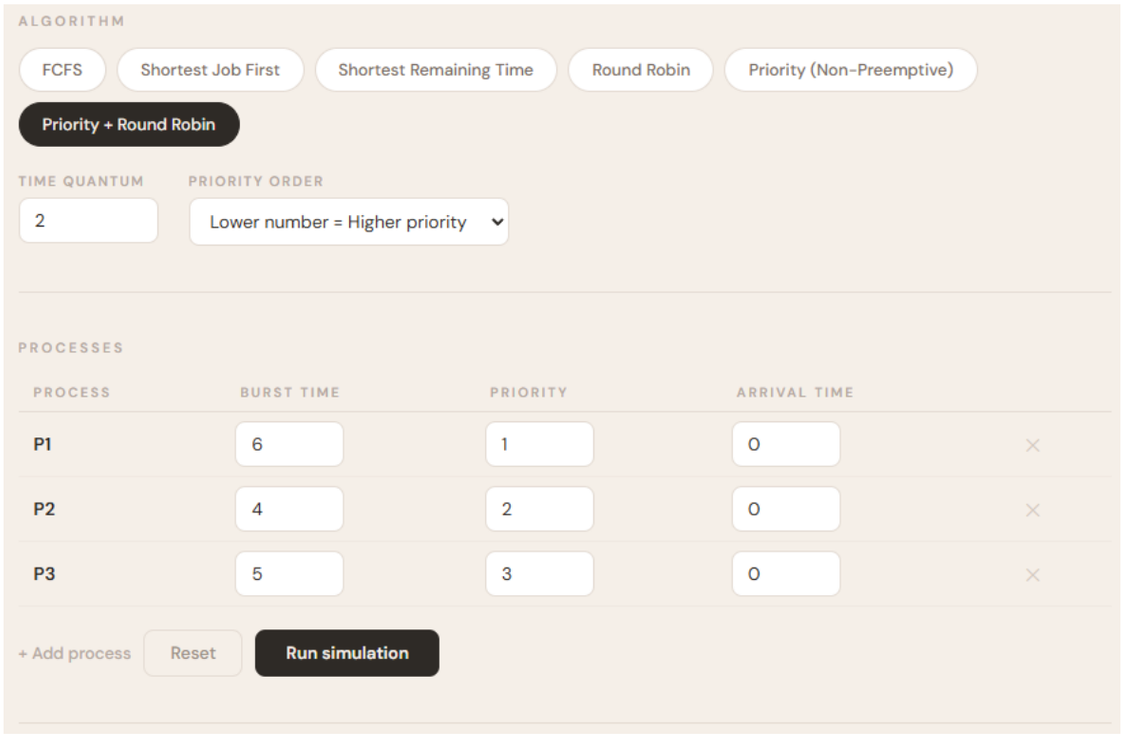

Figure 23. ***Priority with RR Test Case 2 (Input)***

Figure 24. ***Priority with RR Test Case 2 (Output)***

Compared with Test Case 1, this case makes the Round Robin behavior less visible because there is only one process in each priority level. The output is influenced more by strict priority grouping than by within-group rotation, showing that the hybrid algorithm behaves differently depending on how many processes share the same priority.

G. Comparison

Comparing CPU scheduling algorithms should be done using multiple metrics. Each algorithm is good for certain metrics, including waiting time, turnaround time, fairness and responsiveness (González-Rodríguez et al., 2024). In this paper, the simulator demonstrated FCFS was simple but prone to the convoy effect, SJF and SRT were better in terms of waiting time, Round Robin was more fair in terms of CPU allocation and priority-based scheduling algorithms were better for urgent tasks.

The study also shows that there is no algorithm that fits all. The most effective algorithms for wait time may not be fair in CPU sharing, and the fairest algorithms may not have the shortest turnaround time. That's why we compare test cases with Gantt charts and tables in a practical scheduling simulation study.

III. Summary

Key Insights

Implementing the six CPU scheduling algorithms in a working simulator showed that each algorithm has different strengths and limitations depending on the situation. The most important insight gained is that no single scheduling algorithm is universally best, because each one is designed to prioritize a different performance goal such as low waiting time, fairness, responsiveness, or urgency handling.

Performance

The results of the simulator demonstrated that the algorithms differed in their fairness, efficiency and response time. FCFS was easy to implement but was prone to the convoy effect, SJF and SRT were better at minimizing waiting time, Round Robin was more fair for sharing the CPU, and priority-based algorithms were better for dealing with processes with higher priority. These findings demonstrate that an efficient algorithm in terms of waiting time may not be fair and a fair algorithm may not be efficient in terms of turnaround time.

Relevance

The concepts demonstrated by these algorithms remain relevant in modern operating systems, even if real schedulers are more advanced than the textbook versions used in this study. Current systems still apply the same core principles of preemption, priority handling, and fair CPU allocation, which makes understanding these basic algorithms important for learning how real operating systems manage processes.

Challenges Encountered

One of the main challenges in developing the simulator was handling arrival times, preemption, and the correct construction of the Gantt chart, especially for SRT, Round Robin, and Priority Scheduling with Round Robin. This was addressed by carefully updating the ready queue,

tracking remaining burst times step by step, and recording each change in execution order so that the final output remained accurate and easy to understand

IV. Source Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>CPU Scheduling Simulator</title>
  <link
href="https://fonts.googleapis.com/css2?family=Instrument+Serif:ital@0;1&family=DM+Sa
ns:wght@300;400;500;600&display=swap" rel="stylesheet"/>
  <style>
    /* Remove default browser spacing and sizing quirks on all elements */
    *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

    /* ── PAGE BACKGROUND & LAYOUT ──────────────────────────────────────── */
    body {
      background: #f5efe8;           /* warm cream background */
      font-family: 'DM Sans', sans-serif;
      font-weight: 400;
      color: #2e2a26;                /* dark warm-brown text */
      min-height: 100vh;
      display: flex;
      justify-content: center;       /* horizontally center the content */
      align-items: flex-start;       /* pin to top, not vertically centered */
      padding: 48px 20px 80px;
    }

    /* Constrain page width so content doesn't stretch too wide */
    .wrap {
      width: 100%;
      max-width: 860px;
    }

    /* ── HEADINGS ──────────────────────────────────────────────────────── */

    /* Large serif title at the top of the page */
    .page-title {
      font-family: 'Instrument Serif', serif;
      font-size: 2.4rem;
      font-weight: 400;
      color: #2e2a26;
      margin-bottom: 4px;
      letter-spacing: -0.5px;
    }

    /* Small subtitle/description below the title */
    .page-sub {
      font-size: 0.85rem;
      color: #9a8f86;
      margin-bottom: 40px;
    }

    /* Section label (e.g. "Algorithm", "Processes") — small, uppercase, muted */
    h2 {
      font-family: 'DM Sans', sans-serif;
      font-size: 0.68rem;
      font-weight: 600;
      text-transform: uppercase;
      letter-spacing: 2.5px;
      color: #b5a99f;
      margin-bottom: 14px;
```

```
}

/* Thin horizontal rule used to separate sections visually */
.sep {
  border: none;
  border-top: 1px solid #e4dbd3;
  margin: 36px 0;
}

/* ── ALGORITHM SELECTOR ──────────────────────────────────────────── */

/* Wraps the algorithm pill buttons in a flexible row */
.algo-grid {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  margin-bottom: 20px;
}

/* Individual algorithm pill button (default: white, outlined) */
.algo-btn {
  background: #fff;
  border: 1.5px solid #e4dbd3;
  border-radius: 100px;          /* fully rounded = pill shape */
  padding: 8px 18px;
  font-family: 'DM Sans', sans-serif;
  font-size: 0.82rem;
  font-weight: 500;
  color: #7a6e66;
  cursor: pointer;
  transition: background 0.15s, border-color 0.15s, color 0.15s;
  white-space: nowrap;           /* prevent text from wrapping inside the pill */
}
.algo-btn:hover { border-color: #c4b5ab; color: #2e2a26; }

/* Active/selected algorithm pill — dark filled */
.algo-btn.active { background: #2e2a26; border-color: #2e2a26; color: #f5efe8; }

/* ── EXTRA CONTROLS (quantum, priority order) ────────────────────── */

/* Container for optional inputs that appear based on selected algorithm */
.extra-controls {
  display: flex;
  flex-wrap: wrap;
  gap: 24px;
  margin-top: 16px;
}

/* Wraps a label + input pair vertically */
.form-group { display: flex; flex-direction: column; gap: 6px; }

/* Small uppercase label above an input */
label {
  font-size: 0.68rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 1.5px;
  color: #b5a99f;
}

/* Shared style for number inputs and dropdowns */
input[type="number"], select {
  background: #fff;
```

```
      border: 1.5px solid #e4dbd3;
      color: #2e2a26;
      border-radius: 8px;
      padding: 8px 12px;
      font-family: 'DM Sans', sans-serif;
      font-size: 0.88rem;
      outline: none;
      transition: border-color 0.15s;
      width: 110px;
    }
    input[type="number"]:focus, select:focus { border-color: #b5a99f; }

    /* Dropdowns need to be wider to fit the priority option text */
    select { width: auto; min-width: 220px; }

    /* ── PROCESS INPUT TABLE ──────────────────────────────────────── */

    /* Table where users enter burst time, arrival time, etc. per process */
    .process-table {
      width: 100%;
      border-collapse: collapse;     /* removes gaps between cell borders */
      margin-bottom: 18px;
    }

    /* Column header row — small, muted, uppercase */
    .process-table th {
      font-size: 0.66rem;
      font-weight: 600;
      text-transform: uppercase;
      letter-spacing: 2px;
      color: #b5a99f;
      padding: 8px 12px;
      text-align: left;
      border-bottom: 1px solid #e4dbd3;
    }

    /* Each data cell — light bottom border between rows */
    .process-table td { padding: 7px 8px; border-bottom: 1px solid #f0e9e2; }

    /* Constrain number inputs inside the table */
    .process-table input { width: 86px; }

    /* The process ID cell (e.g. "P1") — bold and slightly larger */
    .pid-cell { font-weight: 600; font-size: 0.9rem; color: #2e2a26; padding: 7px
12px !important; }

    /* ── ACTION BUTTONS ──────────────────────────────────────────── */

    /* Row of buttons below the process table */
    .action-row { display: flex; gap: 10px; align-items: center; flex-wrap: wrap; }

    /* Primary CTA — dark filled "Run simulation" button */
    .btn-run {
      background: #2e2a26; color: #f5efe8; border: none; border-radius: 8px;
      padding: 10px 24px; font-family: 'DM Sans', sans-serif; font-size: 0.85rem;
      font-weight: 600; cursor: pointer; transition: opacity 0.15s;
    }
    .btn-run:hover { opacity: 0.8; }

    /* Secondary outlined button — "Reset" */
    .btn-ghost {
      background: transparent; border: 1.5px solid #e4dbd3; color: #9a8f86;
      border-radius: 8px; padding: 9px 20px; font-family: 'DM Sans', sans-serif;
```

```
      font-size: 0.85rem; font-weight: 500; cursor: pointer; transition: border-color
0.15s, color 0.15s;
    }
    .btn-ghost:hover { border-color: #b5a99f; color: #2e2a26; }

    /* Borderless text-style button — "+ Add process" */
    .btn-add {
      background: transparent; border: none; color: #b5a99f;
      font-family: 'DM Sans', sans-serif; font-size: 0.82rem; font-weight: 500;
      cursor: pointer; padding: 6px 0; transition: color 0.15s;
    }
    .btn-add:hover { color: #2e2a26; }

    /* Small ✕ delete button on each process row */
    .btn-del {
      background: transparent; border: none; color: #d4c9c0;
      cursor: pointer; font-size: 0.9rem; padding: 4px 8px; transition: color 0.15s;
    }
    .btn-del:hover { color: #c4847a; } /* turns reddish on hover as a warning */

    /* ── UTILITY ──────────────────────────────────────────────────────── */

    /* Inline validation error message shown below the process table */
    .error-msg { font-size: 0.8rem; color: #c4847a; margin-top: 10px; display: none;
}

    /* Hides elements that should only appear conditionally (e.g. priority column) */
    .hidden { display: none !important; }

    /* ── OUTPUT SECTION ──────────────────────────────────────────────── */

    /* Italic serif name of the selected algorithm shown above results */
    .result-algo-name {
      font-family: 'Instrument Serif', serif;
      font-size: 1.3rem;
      font-style: italic;
      color: #7a6e66;
      margin-bottom: 22px;
    }

    /* ── GANTT CHART ─────────────────────────────────────────────────── */

    /* Allows the Gantt to scroll horizontally on small screens */
    .gantt-wrapper { overflow-x: auto; padding-bottom: 6px; }

    /* Row of colored blocks, one per process run */
    .gantt-chart {
      display: flex;
      align-items: stretch;
      min-width: max-content;        /* prevents wrapping — the chart is one
continuous row */
      margin-bottom: 4px;
      border-radius: 10px;
      overflow: hidden;
    }

    /* A single colored block in the Gantt chart */
    .gantt-block {
      display: flex; align-items: center; justify-content: center;
      min-width: 44px; height: 48px;
      font-family: 'DM Sans', sans-serif; font-size: 0.78rem; font-weight: 600;
      color: rgba(0,0,0,0.4);        /* semi-transparent text so it works on any
pastel bg */
```

```
      transition: filter 0.15s;
    }
    .gantt-block:hover { filter: brightness(0.93); }

    /* IDLE block — shown when CPU is waiting for a process to arrive */
    .gantt-block.idle { background: #ede7df !important; color: #b5a99f; }

    /* The time tick numbers below the Gantt chart */
    .gantt-timeline { display: flex; min-width: max-content; }
    .gantt-tick { font-size: 0.62rem; color: #b5a99f; text-align: right; padding-
right: 3px; }

    /* ── RESULTS TABLE ──────────────────────────────────────────────── */

    /* Table showing per-process waiting time, turnaround, etc. */
    .results-table { width: 100%; border-collapse: collapse; font-size: 0.85rem; }

    /* Column headers — right-aligned (except the first "Process" column) */
    .results-table th {
      font-size: 0.64rem; font-weight: 600; text-transform: uppercase; letter-
spacing: 2px;
      color: #b5a99f; padding: 9px 14px; text-align: right; border-bottom: 1px solid
#e4dbd3;
    }
    .results-table th:first-child { text-align: left; }

    /* Data cells — right-aligned numbers */
    .results-table td { padding: 9px 14px; text-align: right; border-bottom: 1px
solid #f0e9e2; color: #4a4440; }
    .results-table td:first-child { text-align: left; font-weight: 600; }

    /* Remove bottom border from the very last row */
    .results-table tr:last-child td { border-bottom: none; }

    /* Average row at the bottom — slightly muted to distinguish from process rows */
    .results-table .avg-row td { color: #9a8f86; font-size: 0.78rem; border-top: 1px
solid #e4dbd3; font-weight: 400; }
    .results-table .avg-row td:first-child { text-transform: uppercase; letter-
spacing: 1.5px; font-size: 0.64rem; }

    /* ── METRIC SUMMARY CARDS ──────────────────────────────────────── */

    /* Two cards side by side showing avg waiting time and avg turnaround time */
    .metric-row { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; margin-
top: 28px; }

    /* Individual metric card — white box with border */
    .metric-card {
      background: #fff; border: 1.5px solid #e4dbd3; border-radius: 12px; padding:
20px 22px;
    }

    /* Small label above the number (e.g. "Avg Waiting Time") */
    .metric-label { font-size: 0.66rem; font-weight: 600; text-transform: uppercase;
letter-spacing: 2px; color: #b5a99f; margin-bottom: 6px; }

    /* The large number itself */
    .metric-value { font-family: 'Instrument Serif', serif; font-size: 2.4rem; color:
#2e2a26; line-height: 1; }

    /* ── RESPONSIVE ─────────────────────────────────────────────────── */
    @media (max-width: 560px) {
      /* Stack metric cards vertically on small screens */
```

```
      .metric-row { grid-template-columns: 1fr; }
      /* Slightly smaller title on mobile */
      .page-title { font-size: 1.8rem; }
    }
  </style>
</head>
<body>
<!-- ── MAIN WRAPPER — everything lives inside this one div ──────────────── -->
<div class="wrap">

  <!-- Page title and course subtitle -->
  <p class="page-title">CPU Scheduling Simulator</p>
  <p class="page-sub">Operating Systems — Process Scheduling Case Study</p>

  <!-- ── SECTION 1: ALGORITHM SELECTION ──────────────────────────────────── -->
  <h2>Algorithm</h2>

  <!-- Pill buttons — clicking one sets the active algorithm -->
  <div class="algo-grid">
    <button class="algo-btn active" onclick="selectAlgo('FCFS', this)">FCFS</button>
    <button class="algo-btn" onclick="selectAlgo('SJF', this)">Shortest Job
First</button>
    <button class="algo-btn" onclick="selectAlgo('SRT', this)">Shortest Remaining
Time</button>
    <button class="algo-btn" onclick="selectAlgo('RR', this)">Round Robin</button>
    <button class="algo-btn" onclick="selectAlgo('PRIORITY', this)">Priority (Non-
Preemptive)</button>
    <button class="algo-btn" onclick="selectAlgo('PRIORITY_RR', this)">Priority +
Round Robin</button>
  </div>

  <!-- Extra inputs that only appear for certain algorithms -->
  <div class="extra-controls">

    <!-- Shown only for Round Robin and Priority + RR -->
    <div id="quantum-group" class="form-group hidden">
      <label>Time Quantum</label>
      <input type="number" id="quantum" value="2" min="1" />
    </div>

    <!-- Shown only for Priority and Priority + RR -->
    <div id="priority-order-group" class="form-group hidden">
      <label>Priority Order</label>
      <select id="priority-order">
        <option value="lower">Lower number = Higher priority</option>
        <option value="higher">Higher number = Higher priority</option>
      </select>
    </div>

  </div>

  <hr class="sep" />

  <!-- ── SECTION 2: PROCESS INPUT ────────────────────────────────────────── -->
  <h2>Processes</h2>

  <!-- Each row represents one process; rows are added/removed dynamically via JS -->
  <table class="process-table">
    <thead>
      <tr>
        <th>Process</th>
        <th>Burst Time</th>
```

```
        <!-- Priority column is hidden unless a priority-based algorithm is selected
-->
        <th id="priority-header" class="hidden">Priority</th>
        <th>Arrival Time</th>
        <th></th> <!-- empty column for the delete button -->
      </tr>
    </thead>
    <!-- Process rows are injected here by addProcessRow() -->
    <tbody id="process-tbody"></tbody>
  </table>

  <!-- Buttons: add a new row, reset everything, or run the simulation -->
  <div class="action-row">
    <button class="btn-add" onclick="addProcessRow()">+ Add process</button>
    <button class="btn-ghost" onclick="resetAll()">Reset</button>
    <button class="btn-run" onclick="runScheduler()">Run simulation</button>
  </div>

  <!-- Validation error message (hidden by default, shown if input is invalid) -->
  <div class="error-msg" id="error-msg"></div>

  <!-- ── SECTION 3: OUTPUT (hidden until simulation is run) ──────────────── -->
  <div id="output-section" class="hidden">
    <hr class="sep" />

    <!-- Name of the algorithm that was just run (filled in by JS) -->
    <p class="result-algo-name" id="algo-label"></p>

    <!-- ── RESULTS TABLE ──────────────────────────────────────────────────── -->
    <h2>Process Summary</h2>
    <table class="results-table">
      <thead>
        <tr>
          <th>Process</th>
          <th>Burst</th>
          <!-- Priority column only visible for priority-based algorithms -->
          <th id="result-priority-header" class="hidden">Priority</th>
          <th>Arrival</th>
          <th>Finish</th>
          <th>Waiting</th>
          <th>Turnaround</th>
        </tr>
      </thead>
      <!-- Per-process result rows + average row are injected here by JS -->
      <tbody id="results-tbody"></tbody>
    </table>

    <!-- ── METRIC SUMMARY CARDS ──────────────────────────────────────────── -->
    <div class="metric-row">
      <div class="metric-card">
        <div class="metric-label">Avg Waiting Time</div>
        <!-- Value filled in by JS after simulation -->
        <div class="metric-value" id="avg-wt">—</div>
      </div>
      <div class="metric-card">
        <div class="metric-label">Avg Turnaround Time</div>
        <!-- Value filled in by JS after simulation -->
        <div class="metric-value" id="avg-tat">—</div>
      </div>
    </div>

    <hr class="sep" />
```

```
    <!-- ── GANTT CHART ──────────────────────────────────────────────── -->
    <h2>Gantt Chart</h2>
    <div class="gantt-wrapper">
      <!-- Colored blocks are injected here by JS, one per timeline segment -->
      <div class="gantt-chart" id="gantt-chart"></div>
      <!-- Time tick numbers below the chart (0, 2, 5, 8 ...) -->
      <div class="gantt-timeline" id="gantt-timeline"></div>
    </div>

  </div>
  <!-- end #output-section -->

</div>
<!-- end .wrap -->
<script>
var currentAlgo = "FCFS";
var processCount = 0;
var PROCESS_COLORS = [
  "#b8d4f0","#a8dfbe","#f5c6a0","#f0b8c4","#c8b8e8",
  "#a8d8d4","#f5d4a0","#d4c4f0","#b8e4c0","#f0c8b0"
];

window.onload = function() { addProcessRow(); addProcessRow(); addProcessRow(); };

function selectAlgo(algo, btn) {
  currentAlgo = algo;
  document.querySelectorAll(".algo-btn").forEach(function(b){
b.classList.remove("active"); });
  btn.classList.add("active");

  var qg = document.getElementById("quantum-group");
  (algo==="RR"||algo==="PRIORITY_RR") ? qg.classList.remove("hidden") :
qg.classList.add("hidden");

  var pg = document.getElementById("priority-order-group");
  var ph = document.getElementById("priority-header");

  if (algo==="PRIORITY"||algo==="PRIORITY_RR") {
    pg.classList.remove("hidden");
    ph.classList.remove("hidden");
  }

  else {
    pg.classList.add("hidden");
    ph.classList.add("hidden");
  }

  document.querySelectorAll(".priority-cell").forEach(function(c){
    (algo==="PRIORITY"||algo==="PRIORITY_RR") ? c.classList.remove("hidden") :
c.classList.add("hidden");
  });
}

function addProcessRow() {
  processCount++;
  var pid = "P" + processCount;
  var row = document.createElement("tr");
  row.id = "row-" + pid;
  var ph = (currentAlgo==="PRIORITY"||currentAlgo==="PRIORITY_RR") ? "" : "hidden";
  row.innerHTML =
    '<td class="pid-cell">'+pid+'</td>'+
    '<td><input type="number" id="bt-'+pid+'" value="'+processCount+'"
min="1"/></td>'+
```

```
    '<td class="priority-cell '+ph+'"><input type="number" id="pr-'+pid+'"
value="'+processCount+'" min="1"/></td>'+
    '<td><input type="number" id="at-'+pid+'" value="0" min="0"/></td>'+
    '<td><button class="btn-del" onclick="removeRow(\''+pid+'\')">✕</button></td>';
  document.getElementById("process-tbody").appendChild(row);
}

function removeRow(pid) {
  if (document.querySelectorAll("#process-tbody tr").length <= 3) {
showError("Minimum of 3 processes required."); return; }
  showError(""); var r = document.getElementById("row-"+pid); if(r) r.remove();
}

function resetAll() {
  processCount = 0;
  document.getElementById("process-tbody").innerHTML = "";
  document.getElementById("output-section").classList.add("hidden");
  showError(""); addProcessRow(); addProcessRow(); addProcessRow();
}

function showError(msg) {
  var el = document.getElementById("error-msg");
  el.textContent = msg; el.style.display = msg ? "block" : "none";
}

function getProcesses() {
  var list = [];
  document.querySelectorAll("#process-tbody tr").forEach(function(row) {
    var pid = row.querySelector(".pid-cell").textContent;
    var at = parseInt(document.getElementById("at-"+pid).value);
    var bt = parseInt(document.getElementById("bt-"+pid).value);
    var pri = document.getElementById("pr-"+pid);
    var pr = pri ? parseInt(pri.value) : 0;
    if (!isNaN(at) && !isNaN(bt) && bt>=1)
list.push({pid:pid,at:at,bt:bt,priority:pr});
  });
  return list;
}

function runScheduler() {
  showError("");
  var processes = getProcesses();
  if (processes.length < 2) { showError("Please enter at least 2 valid processes.");
return; }
  var quantum = parseInt(document.getElementById("quantum").value)||2;
  var priorityOrder = document.getElementById("priority-order").value;
  var timeline=[], result=[];
  if      (currentAlgo==="FCFS")        timeline=runFCFS(processes,result);
  else if (currentAlgo==="SJF")         timeline=runSJF(processes,result);
  else if (currentAlgo==="SRT")         timeline=runSRT(processes,result);
  else if (currentAlgo==="RR")          timeline=runRR(processes,quantum,result);
  else if
(currentAlgo==="PRIORITY")    timeline=runPriority(processes,priorityOrder,result);
  else if (currentAlgo==="PRIORITY_RR")
timeline=runPriorityRR(processes,quantum,priorityOrder,result);
  drawOutput(timeline, result, processes);
}

// ── HELPERS ──────────────────────────────────────────────────────

// Add a Gantt block to the timeline
function addBlock(tl, pid, start, end) {
  tl.push({ pid: pid, start: start, end: end });
```

```
}

// If the CPU has nothing to run, add an IDLE block until the next arrival
function addIdleIfNeeded(tl, t, processes, excludeList) {
  var pending = processes.filter(function(p) { return excludeList.indexOf(p.pid) ===
-1; });
  var nextArrival = pending.reduce(function(min, p) { return p.at < min ? p.at : min;
}, Infinity);
  if (nextArrival > t) addBlock(tl, "IDLE", t, nextArrival);
  return nextArrival;
}

// Get all processes that have arrived and haven't finished yet
function getReady(processes, t, excludeList) {
  return processes.filter(function(p) { return p.at <= t &&
excludeList.indexOf(p.pid) === -1; });
}

// Record a completed process into the result array
function recordResult(result, p, finishTime) {
  result.push({ pid: p.pid, at: p.at, bt: p.bt, priority: p.priority, ft: finishTime
});
}

// Merge back-to-back blocks of the same process into one
function mergeTimeline(tl) {
  if (!tl.length) return [];
  var merged = [tl[0]];
  for (var i = 1; i < tl.length; i++) {
    var last = merged[merged.length - 1];
    var curr = tl[i];
    // If same process continues right after, just extend the previous block
    if (last.pid === curr.pid && last.end === curr.start) {
      last.end = curr.end;
    } else {
      merged.push(curr);
    }
  }
  return merged;
}

// ─── ALGORITHM 1: FCFS ──────────────────────────────────────────────
// Run processes in the order they arrive. No preemption.

function runFCFS(processes, result) {
  var tl = [];
  var t = 0;

  // Sort by arrival time so we process whoever came first
  var sorted = processes.slice().sort(function(a, b) { return a.at - b.at; });

  for (var i = 0; i < sorted.length; i++) {
    var p = sorted[i];

    // CPU is idle while waiting for this process to arrive
    if (t < p.at) {
      addBlock(tl, "IDLE", t, p.at);
      t = p.at;
    }

    // Run the process to completion
    addBlock(tl, p.pid, t, t + p.bt);
    recordResult(result, p, t + p.bt);
```

```
    t += p.bt;
  }

  return tl;
}

// ── ALGORITHM 2: SJF ──────────────────────────────────────────────
// Among arrived processes, always pick the one with the shortest burst. No
preemption.

function runSJF(processes, result) {
  var tl = [];
  var t = 0;
  var done = [];

  while (done.length < processes.length) {
    var ready = getReady(processes, t, done);

    if (!ready.length) {
      // Nothing ready — jump to next arrival
      t = addIdleIfNeeded(tl, t, processes, done);
      continue;
    }

    // Pick the shortest job among ready processes
    ready.sort(function(a, b) { return a.bt - b.bt; });
    var p = ready[0];

    addBlock(tl, p.pid, t, t + p.bt);
    recordResult(result, p, t + p.bt);
    t += p.bt;
    done.push(p.pid);
  }

  return tl;
}

// ── ALGORITHM 3: SRT ──────────────────────────────────────────────
// Preemptive version of SJF. At every tick, the process with the least
// remaining time runs. If a shorter job arrives, it takes over immediately.

function runSRT(processes, result) {
  var tl = [];
  var t = 0;
  var remaining = {};      // remaining burst time per process
  var finishTime = {};
  var finished = [];
  var currentPid = null;   // which process is running right now
  var blockStart = 0;      // when the current run started

  // Initialize remaining time for each process
  processes.forEach(function(p) { remaining[p.pid] = p.bt; });

  while (finished.length < processes.length) {
    var ready = getReady(processes, t, finished);

    if (!ready.length) {
      // Save any current block before going idle
      if (currentPid) {
        addBlock(tl, currentPid, blockStart, t);
        currentPid = null;
      }
      t = addIdleIfNeeded(tl, t, processes, finished);
```

```
      continue;
    }

    // Pick the process with the shortest remaining time
    ready.sort(function(a, b) { return remaining[a.pid] - remaining[b.pid]; });
    var p = ready[0];

    // If a different process takes over, close the previous block
    if (p.pid !== currentPid) {
      if (currentPid) addBlock(tl, currentPid, blockStart, t);
      currentPid = p.pid;
      blockStart = t;
    }

    // Run for 1 tick
    remaining[p.pid]--;
    t++;

    // If this process just finished, close its block
    if (remaining[p.pid] === 0) {
      addBlock(tl, p.pid, blockStart, t);
      finishTime[p.pid] = t;
      finished.push(p.pid);
      currentPid = null;
    }
  }

  processes.forEach(function(p) { recordResult(result, p, finishTime[p.pid]); });
  return mergeTimeline(tl);
}

// ── ALGORITHM 4: ROUND ROBIN ──────────────────────────────────────
// Each process gets a fixed time slice (quantum). If not done, it goes
// to the back of the queue.

function runRR(processes, quantum, result) {
  var tl = [];
  var t = 0;
  var remaining = {};
  var finishTime = {};
  var queue = [];       // order in which processes wait to run
  var arrived = [];     // tracks who has joined the queue already
  var finished = [];

  processes.forEach(function(p) { remaining[p.pid] = p.bt; });

  // Sort by arrival so we can enqueue in the right order
  var sorted = processes.slice().sort(function(a, b) { return a.at - b.at; });

  // Enqueue any processes that arrive at time 0
  sorted.forEach(function(p) {
    if (p.at <= t && arrived.indexOf(p.pid) === -1) {
      queue.push(p.pid);
      arrived.push(p.pid);
    }
  });

  while (finished.length < processes.length) {
    // If queue is empty, jump ahead to next arrival
    if (!queue.length) {
      var next = sorted.find(function(p) { return arrived.indexOf(p.pid) === -1; });
      if (!next) break;
      addBlock(tl, "IDLE", t, next.at);
```

```
      t = next.at;
      queue.push(next.pid);
      arrived.push(next.pid);
    }

    var pid = queue.shift();
    var runTime = Math.min(quantum, remaining[pid]);

    // Run this process for its time slice
    addBlock(tl, pid, t, t + runTime);
    t += runTime;
    remaining[pid] -= runTime;

    // Enqueue any new arrivals that showed up during this slice
    sorted.forEach(function(p) {
      if (p.at <= t && arrived.indexOf(p.pid) === -1) {
        queue.push(p.pid);
        arrived.push(p.pid);
      }
    });

    if (remaining[pid] === 0) {
      // Process is done
      finishTime[pid] = t;
      finished.push(pid);
    } else {
      // Not done — send to the back of the queue
      queue.push(pid);
    }
  }

  processes.forEach(function(p) { recordResult(result, p, finishTime[p.pid]); });
  return tl;
}

// ─── ALGORITHM 5: PRIORITY ──────────────────────────────────────────
// Among arrived processes, pick the one with the best priority. No preemption.

function runPriority(processes, priorityOrder, result) {
  var tl = [];
  var t = 0;
  var done = [];

  // Sort function: lower number = higher priority, or vice versa
  function byPriority(a, b) {
    return priorityOrder === "lower" ? a.priority - b.priority : b.priority -
a.priority;
  }

  while (done.length < processes.length) {
    var ready = getReady(processes, t, done);

    if (!ready.length) {
      t = addIdleIfNeeded(tl, t, processes, done);
      continue;
    }

    // Pick highest-priority process
    ready.sort(byPriority);
    var p = ready[0];

    addBlock(tl, p.pid, t, t + p.bt);
    recordResult(result, p, t + p.bt);
```

```
    t += p.bt;
    done.push(p.pid);
  }

  return tl;
}

// ─── ALGORITHM 6: PRIORITY + ROUND ROBIN ──────────────────────────────
// Like Round Robin, but the queue is re-sorted by priority each cycle.

function runPriorityRR(processes, quantum, priorityOrder, result) {
  var tl = [];
  var t = 0;
  var remaining = {};
  var finishTime = {};
  var queue = [];
  var arrived = [];
  var finished = [];

  processes.forEach(function(p) { remaining[p.pid] = p.bt; });

  var sorted = processes.slice().sort(function(a, b) { return a.at - b.at; });

  // Helper: get a process's priority value by pid
  function getPriority(pid) {
    var p = processes.find(function(x) { return x.pid === pid; });
    return p ? p.priority : 0;
  }

  // Sort function for the queue
  function byPriority(a, b) {
    return priorityOrder === "lower" ? getPriority(a) - getPriority(b) :
getPriority(b) - getPriority(a);
  }

  // Enqueue processes already arrived at t=0
  sorted.forEach(function(p) {
    if (p.at <= t && arrived.indexOf(p.pid) === -1) {
      queue.push(p.pid);
      arrived.push(p.pid);
    }
  });

  while (finished.length < processes.length) {
    // Sort queue by priority before picking next process
    queue.sort(byPriority);

    if (!queue.length) {
      var next = sorted.find(function(p) { return arrived.indexOf(p.pid) === -1; });
      if (!next) break;
      addBlock(tl, "IDLE", t, next.at);
      t = next.at;
      queue.push(next.pid);
      arrived.push(next.pid);
    }

    var pid = queue.shift();
    var runTime = Math.min(quantum, remaining[pid]);

    addBlock(tl, pid, t, t + runTime);
    t += runTime;
    remaining[pid] -= runTime;
```

```
    // Enqueue new arrivals
    sorted.forEach(function(p) {
      if (p.at <= t && arrived.indexOf(p.pid) === -1) {
        queue.push(p.pid);
        arrived.push(p.pid);
      }
    });

    if (remaining[pid] === 0) {
      finishTime[pid] = t;
      finished.push(pid);
    } else {
      queue.push(pid);
    }
  }

  processes.forEach(function(p) { recordResult(result, p, finishTime[p.pid]); });
  return tl;
}

function computeMetrics(result) {
  result.forEach(function(r){r.tat=r.ft-r.at;r.wt=r.tat-r.bt;});
  var n=result.length;
  return {
    avgWT:(result.reduce(function(s,r){return s+r.wt;},0)/n).toFixed(2),
    avgTAT:(result.reduce(function(s,r){return s+r.tat;},0)/n).toFixed(2)
  };
}

function buildColorMap(processes) {
  var map={};
  processes.forEach(function(p,i){map[p.pid]=PROCESS_COLORS[i%PROCESS_COLORS.length];
});
  return map;
}

function drawOutput(timeline, result, processes) {
  computeMetrics(result);
  var cm=buildColorMap(processes);
  var out=document.getElementById("output-section");
  out.classList.remove("hidden");

  var names={
    FCFS:"First-Come, First-Served (FCFS)",SJF:"Shortest Job First (SJF)",
    SRT:"Shortest Remaining Time (SRT)",RR:"Round Robin (RR)",
    PRIORITY:"Priority Scheduling",PRIORITY_RR:"Priority Scheduling with Round Robin"
  };
  document.getElementById("algo-label").textContent=names[currentAlgo]||currentAlgo;

  var gc=document.getElementById("gantt-chart"),gt=document.getElementById("gantt-
timeline");
  gc.innerHTML="";gt.innerHTML="";
  var uw=44;
  timeline.forEach(function(b,i) {
    var w=Math.max(uw,(b.end-b.start)*uw);
    var d=document.createElement("div");
    d.className="gantt-block";d.style.width=w+"px";d.style.minWidth=w+"px";
    if(b.pid==="IDLE"){d.classList.add("idle");d.textContent="—";}
    else{d.style.background=cm[b.pid]||"#ddd";d.textContent=b.pid;}
    d.title=b.pid+" ["+b.start+" → "+b.end+"]";gc.appendChild(d);
    if(i===0){var s=document.createElement("div");s.className="gantt-
tick";s.style.width="0px";s.textContent=b.start;gt.appendChild(s);}
```

```
    var tk=document.createElement("div");tk.className="gantt-
tick";tk.style.width=w+"px";tk.style.textAlign="right";tk.textContent=b.end;gt.append
Child(tk);
  });

  var showP=currentAlgo==="PRIORITY"||currentAlgo==="PRIORITY_RR";
  var rph=document.getElementById("result-priority-header");
  showP?rph.classList.remove("hidden"):rph.classList.add("hidden");

  var tbody=document.getElementById("results-tbody");tbody.innerHTML="";
  result.forEach(function(r) {
    var row=document.createElement("tr");
    var dot='<span style="display:inline-block;width:9px;height:9px;border-
radius:50%;background:'+(cm[r.pid]||"#ddd")+';margin-right:8px;vertical-
align:middle"></span>';
    var pc=showP?'<td>'+(r.priority!==undefined?r.priority:'—')+'</td>':'';
    row.innerHTML='<td>'+dot+r.pid+'</td><td>'+r.bt+'</td>'+pc+'<td>'+r.at+'</td><td>
'+r.ft+'</td><td>'+r.wt+'</td><td>'+r.tat+'</td>';
    tbody.appendChild(row);
  });
  var avgs=computeMetrics(result);
  var avgRow=document.createElement("tr");avgRow.className="avg-row";
  var ac=showP?5:4;
  avgRow.innerHTML='<td
colspan="'+ac+'">Average</td><td>'+avgs.avgWT+'</td><td>'+avgs.avgTAT+'</td>';
  tbody.appendChild(avgRow);
  document.getElementById("avg-wt").textContent=avgs.avgWT;
  document.getElementById("avg-tat").textContent=avgs.avgTAT;
  out.scrollIntoView({behavior:"smooth"});
}
</script>
</body>
</html>
```

Figure 25. ***CPU Scheduling Algorithms Source Code***

HTML

The semantic structure of the simulator is defined by the HTML, dividing it into separate sections, including the header with a title of CPU Scheduling Simulator, an area of selecting an algorithm, an area of process input, an area of process summary, and an area of Gantt chart.

It assigns each logical component of the simulator with explicit IDs and types of elements that will then be read by the JavaScript to get user input and inject results into the DOM. The buttons of the “Algorithm” field is based on the list of scheduling methods available in the UI. Lastly, the Process Summary and Gantt Chart sections are associated with the output tables and visualization that show the completion time, waiting time, turnaround time, and the execution timeline.

CSS

CSS section manages the appearance, readability, and layout characteristics of the simulator. Its presentation style is minimalistic warm, appearing clean and academically polished as opposed to being too technical.

The algorithm buttons are designed to be in the form of pill-shaped controls, which visually distinguish between the currently running algorithm and the inactive alternatives. The significance of this is that the simulator can only accept one single selected scheduling method at a time and that the CSS can support that reasoning by making the active state that is immediately recognizable.

The CSS of the Gantt chart is particularly meaningful since it provides the opportunity to make the timeline operate as a visual implementation plan of CPU activity. These blocks are packed in a horizontal flex container, so that the execution of processes is modeled as a continuous stream of adjacent colored blocks.

JavaScript

JavaScript section is the computational core of the simulator since it is in charge of interface behavior, dynamically manages process rows, implements the CPU scheduling algorithms, calculates the finish, waiting and turnaround times, and finally renders the result table as well as the Gantt chart. JavaScript is the processing layer which converts input of the user into the output of the simulation algorithm.

The script starts with defining global variables that store the current algorithm being used, process rows that have been created, and the color palette of the visualization. It also loads the page by automatically creating three process rows when the page is loaded, so that the simulator can be immediately used.

```
var currentAlgo = "FCFS";
var processCount = 0;
var PROCESS_COLORS = [
  "#b8d4f0","#a8dfbe","#f5c6a0","#f0b8c4","#c8b8e8",
  "#a8d8d4","#f5d4a0","#d4c4f0","#b8e4c0","#f0c8b0"
];

window.onload = function() { addProcessRow(); addProcessRow(); addProcessRow(); };
```

Figure 26. ***Initialization Phase***

This initialization phase is important because it establishes the baseline system state before the user interacts with the application. The use of predefined colors also demonstrates that the simulator is designed not only to calculate process execution but also to communicate it visually through consistent color mapping.

```
function selectAlgo(algo, btn) {
  currentAlgo = algo;
  document.querySelectorAll(".algo-btn").forEach(function(b){
b.classList.remove("active"); });
  btn.classList.add("active");
  var qg = document.getElementById("quantum-group");
  (algo==="RR"||algo==="PRIORITY_RR") ? qg.classList.remove("hidden") :
qg.classList.add("hidden");
  var pg = document.getElementById("priority-order-group");
  var ph = document.getElementById("priority-header");
  if (algo==="PRIORITY"||algo==="PRIORITY_RR") { pg.classList.remove("hidden");
ph.classList.remove("hidden"); }
  else { pg.classList.add("hidden"); ph.classList.add("hidden"); }
  document.querySelectorAll(".priority-cell").forEach(function(c){
    (algo==="PRIORITY"||algo==="PRIORITY_RR") ? c.classList.remove("hidden") :
c.classList.add("hidden");
  });
}
```

Figure 27. ***selectAlgo()***

The selectAlgo method handles the modifications in the chosen scheduling algorithm and reflects the interface on them. It primarily functions to match the visual state of the algorithm buttons with the logical state of the application, as well as to reveal or hide some additional controls, such as the time quantum and the priority order depending on the selected algorithm.

```
function addProcessRow() {
  processCount++;
  var pid = "P" + processCount;
  var row = document.createElement("tr");
  row.id = "row-" + pid;
  var ph = (currentAlgo==="PRIORITY"||currentAlgo==="PRIORITY_RR") ? "" : "hidden";
  row.innerHTML =
    '<td class="pid-cell">'+pid+'</td>'+
    '<td><input type="number" id="bt-'+pid+'" value="'+processCount+'"
min="1"/></td>'+
```

```
    '<td class="priority-cell '+ph+'"><input type="number" id="pr-'+pid+'"
value="'+processCount+'" min="1"/></td>'+
    '<td><input type="number" id="at-'+pid+'" value="0" min="0"/></td>'+
    '<td><button class="btn-del" onclick="removeRow(\''+pid+'\')">X</button></td>';
  document.getElementById("process-tbody").appendChild(row);
}
```

Figure 28. ***addProcessRow()***

```
function getProcesses() {
  var list = [];
  document.querySelectorAll("#process-tbody tr").forEach(function(row) {
    var pid = row.querySelector(".pid-cell").textContent;
    var at = parseInt(document.getElementById("at-"+pid).value);
    var bt = parseInt(document.getElementById("bt-"+pid).value);
    var pri = document.getElementById("pr-"+pid);
    var pr = pri ? parseInt(pri.value) : 0;
    if (!isNaN(at) && !isNaN(bt) && bt>=1)
list.push({pid:pid,at:at,bt:bt,priority:pr});
  });
  return list;
}
```

Figure 29. ***getProcesses()***

The addProcessRow and getProcesses functions collaborate to enable dynamic process management. The former takes the current table contents, creates a new table row with burst time, arrival time, and optional priority inputs, and the latter reads the current table contents and transforms them into structured JavaScript objects that can be processed by an algorithm.

```
function runScheduler() {
  showError("");
  var processes = getProcesses();
  if (processes.length < 2) { showError("Please enter at least 2 valid processes.");
return; }
  var quantum = parseInt(document.getElementById("quantum").value)||2;
  var priorityOrder = document.getElementById("priority-order").value;
  var timeline=[], result=[];
  if      (currentAlgo==="FCFS")        timeline=runFCFS(processes,result);
  else if (currentAlgo==="SJF")         timeline=runSJF(processes,result);
  else if (currentAlgo==="SRT")         timeline=runSRT(processes,result);
  else if (currentAlgo==="RR")          timeline=runRR(processes,quantum,result);
  else if
(currentAlgo==="PRIORITY")    timeline=runPriority(processes,priorityOrder,result);
  else if (currentAlgo==="PRIORITY_RR")
timeline=runPriorityRR(processes,quantum,priorityOrder,result);
  drawOutput(timeline, result, processes);
}
```

Figure 30. ***runScheduler()***

The simulation has a central control point, the runScheduler function that sends execution to the appropriate scheduling algorithm depending on the selected mode. This dispatcher pattern

is a highly-structured design option since it keeps all scheduling methods separate in their functions as opposed to placing all logic in a single large code.

```
function runFCFS(processes, result) {
  var tl = [];
  var t = 0;

  // Sort by arrival time so we process whoever came first
  var sorted = processes.slice().sort(function(a, b) { return a.at - b.at; });

  for (var i = 0; i < sorted.length; i++) {
    var p = sorted[i];

    // CPU is idle while waiting for this process to arrive
    if (t < p.at) {
      addBlock(tl, "IDLE", t, p.at);
      t = p.at;
    }

    // Run the process to completion
    addBlock(tl, p.pid, t, t + p.bt);
    recordResult(result, p, t + p.bt);
    t += p.bt;
  }

  return tl;
}
```

Figure 31. ***runFCFS()***

The FCFS operation is an operation that first copies the input processes, then sorts the processes based on the arrival time, and then visits the processes one after another. The most important variable is t which is the current simulated time; when the next process arrives later than t, the code will insert an IDLE block before the process is executed.

```
function runSJF(processes, result) {
  var tl = [];
  var t = 0;
  var done = [];

  while (done.length < processes.length) {
    var ready = getReady(processes, t, done);

    if (!ready.length) {
      // Nothing ready — jump to next arrival
      t = addIdleIfNeeded(tl, t, processes, done);
      continue;
    }

    // Pick the shortest job among ready processes
    ready.sort(function(a, b) { return a.bt - b.bt; });
    var p = ready[0];

    addBlock(tl, p.pid, t, t + p.bt);
    recordResult(result, p, t + p.bt);
    t += p.bt;
```

```
    done.push(p.pid);
  }

  return tl;
}
```

Figure 32. ***runSJF()***

A done array is used in the SJF function to keep track of which processes have been already completed. Within the while loop it keeps calling getReady(), and it sorts the set of processes that it has received but that are not yet finished, and selects the one with the lowest burst time.

The ready.sort(function(a, b) { return a.bt - b.bt; }) line is the one in which the scheduler determines the next process to run.

```
function runSRT(processes, result) {
  var tl = [];
  var t = 0;
  var remaining = {};      // remaining burst time per process
  var finishTime = {};
  var finished = [];
  var currentPid = null;   // which process is running right now
  var blockStart = 0;      // when the current run started

  // Initialize remaining time for each process
  processes.forEach(function(p) { remaining[p.pid] = p.bt; });

  while (finished.length < processes.length) {
    var ready = getReady(processes, t, finished);

    if (!ready.length) {
      // Save any current block before going idle
      if (currentPid) {
        addBlock(tl, currentPid, blockStart, t);
        currentPid = null;
      }
      t = addIdleIfNeeded(tl, t, processes, finished);
      continue;
    }

    // Pick the process with the shortest remaining time
    ready.sort(function(a, b) { return remaining[a.pid] - remaining[b.pid]; });
    var p = ready[0];

    // If a different process takes over, close the previous block
    if (p.pid !== currentPid) {
      if (currentPid) addBlock(tl, currentPid, blockStart, t);
      currentPid = p.pid;
      blockStart = t;
    }

    // Run for 1 tick
    remaining[p.pid]--;
    t++;

    // If this process just finished, close its block
```

```
    if (remaining[p.pid] === 0) {
      addBlock(tl, p.pid, blockStart, t);
      finishTime[p.pid] = t;
      finished.push(p.pid);
      currentPid = null;
    }
  }

  processes.forEach(function(p) { recordResult(result, p, finishTime[p.pid]); });
  return mergeTimeline(tl);
}
```

Figure 33. ***runSRT()***

The SRT function is more detailed as it maintains an object named remaining to keep track of the remaining burst time per process. It also introduces the currentPid and blockStart into the code to allow closing the timeline block when a process is interrupted, and opening a new block when a new process takes over.

The most important implementation detail here is that the one-tick execution step remains as [p.pid]--; t++;, so that the loop simulates processes in a step by step fashion as opposed to a block execution fashion. It is that option that enables a running process to be interrupted and switched to another one when necessary.

```
function runRR(processes, quantum, result) {
  var tl = [];
  var t = 0;
  var remaining = {};
  var finishTime = {};
  var queue = [];       // order in which processes wait to run
  var arrived = [];     // tracks who has joined the queue already
  var finished = [];

  processes.forEach(function(p) { remaining[p.pid] = p.bt; });

  // Sort by arrival so we can enqueue in the right order
  var sorted = processes.slice().sort(function(a, b) { return a.at - b.at; });

  // Enqueue any processes that arrive at time 0
  sorted.forEach(function(p) {
    if (p.at <= t && arrived.indexOf(p.pid) === -1) {
      queue.push(p.pid);
      arrived.push(p.pid);
    }
  });

  while (finished.length < processes.length) {
    // If queue is empty, jump ahead to next arrival
    if (!queue.length) {
      var next = sorted.find(function(p) { return arrived.indexOf(p.pid) === -1; });
      if (!next) break;
      addBlock(tl, "IDLE", t, next.at);
```

```
      t = next.at;
      queue.push(next.pid);
      arrived.push(next.pid);
    }

    var pid = queue.shift();
    var runTime = Math.min(quantum, remaining[pid]);

    // Run this process for its time slice
    addBlock(tl, pid, t, t + runTime);
    t += runTime;
    remaining[pid] -= runTime;

    // Enqueue any new arrivals that showed up during this slice
    sorted.forEach(function(p) {
      if (p.at <= t && arrived.indexOf(p.pid) === -1) {
        queue.push(p.pid);
        arrived.push(p.pid);
      }
    });

    if (remaining[pid] === 0) {
      // Process is done
      finishTime[pid] = t;
      finished.push(pid);
    } else {
      // Not done — send to the back of the queue
      queue.push(pid);
    }
  }

  processes.forEach(function(p) { recordResult(result, p, finishTime[p.pid]); });
  return tl;
}
```

Figure 34. ***runRR()***

The implementation of Round Robin adds a queue, the main data structure of the function. The next process to be executed is obtained using the queue.shift() operation and then it is either returned to the queue if it is not completed yet or its finish time is written if it completes.

The interesting implementation detail is that it uses Math.min(quantum, remaining[pid]); to make sure that no process is ever allowed to execute for more than the quantum time slice. This line is the one that makes the code not act as a plain list with first in first out execution but as a rotating queue, which is a quantum type list.

```
function runPriority(processes, priorityOrder, result) {
  var tl = [];
  var t = 0;
  var done = [];

  // Sort function: lower number = higher priority, or vice versa
  function byPriority(a, b) {
```

```
    return priorityOrder === "lower" ? a.priority - b.priority : b.priority -
a.priority;
  }

  while (done.length < processes.length) {
    var ready = getReady(processes, t, done);

    if (!ready.length) {
      t = addIdleIfNeeded(tl, t, processes, done);
      continue;
    }

    // Pick highest-priority process
    ready.sort(byPriority);
    var p = ready[0];

    addBlock(tl, p.pid, t, t + p.bt);
    recordResult(result, p, t + p.bt);
    t += p.bt;
    done.push(p.pid);
  }

  return tl;
}
```

Figure 35. ***runPriority()***

The priority function is built similarly to SJF, except that the sorting criterion is priority instead of burst time. The code uses an inner helper byPriority() so that the meaning of "higher priority" can be switched based on the dropdown choice.

```
function runPriorityRR(processes, quantum, priorityOrder, result) {
  var tl = [];
  var t = 0;
  var remaining = {};
  var finishTime = {};
  var queue = [];
  var arrived = [];
  var finished = [];

  processes.forEach(function(p) { remaining[p.pid] = p.bt; });

  var sorted = processes.slice().sort(function(a, b) { return a.at - b.at; });

  // Helper: get a process's priority value by pid
  function getPriority(pid) {
    var p = processes.find(function(x) { return x.pid === pid; });
    return p ? p.priority : 0;
  }

  // Sort function for the queue
  function byPriority(a, b) {
    return priorityOrder === "lower" ? getPriority(a) - getPriority(b) :
getPriority(b) - getPriority(a);
  }

  // Enqueue processes already arrived at t=0
  sorted.forEach(function(p) {
    if (p.at <= t && arrived.indexOf(p.pid) === -1) {
```

```
      queue.push(p.pid);
      arrived.push(p.pid);
    }
  });

  while (finished.length < processes.length) {
    // Sort queue by priority before picking next process
    queue.sort(byPriority);

    if (!queue.length) {
      var next = sorted.find(function(p) { return arrived.indexOf(p.pid) === -1; });
      if (!next) break;
      addBlock(tl, "IDLE", t, next.at);
      t = next.at;
      queue.push(next.pid);
      arrived.push(next.pid);
    }

    var pid = queue.shift();
    var runTime = Math.min(quantum, remaining[pid]);

    addBlock(tl, pid, t, t + runTime);
    t += runTime;
    remaining[pid] -= runTime;

    // Enqueue new arrivals
    sorted.forEach(function(p) {
      if (p.at <= t && arrived.indexOf(p.pid) === -1) {
        queue.push(p.pid);
        arrived.push(p.pid);
      }
    });

    if (remaining[pid] === 0) {
      finishTime[pid] = t;
      finished.push(pid);
    } else {
      queue.push(pid);
    }
  }

  processes.forEach(function(p) { recordResult(result, p, finishTime[p.pid]); });
  return tl;
}
```

Figure 36. ***runPriorityRR()***

The Priority + RR version combines the queue structure of Round Robin with a priority-based sort before each selection. It uses a helper getPriority(pid) to look up each process's priority value by ID, which allows the queue to be reordered even though it only stores process IDs instead of full process objects.

The queue is re-sorted before each selection, so priority affects ordering even though the execution still happens in time slices. In other words, this function is not just "Priority scheduling"

in theory; it is a specific queue-management implementation that merges priority sorting with Round Robin rotation.

```
function computeMetrics(result) {
  result.forEach(function(r){r.tat=r.ft-r.at;r.wt=r.tat-r.bt;});
  var n=result.length;
  return {
    avgWT:(result.reduce(function(s,r){return s+r.wt;},0)/n).toFixed(2),
    avgTAT:(result.reduce(function(s,r){return s+r.tat;},0)/n).toFixed(2)
  };
}
```

Figure 37. ***computeMetrics()***

Once the algorithms are completed, the code calculates turnaround time and waiting time, using the original finish time of each process. The formulas are executed directly from computeMetrics and then the results are used with drawOutput to fill the table and draw the Gantt chart.

V. Repository Link

https://github.com/GianVincentTan/OS-CASE-STUDY-.git

VI. References

Neovise. (2026, March 14). What Is CPU Scheduling in an Operating System? Medium. https://medium.com/@neovise/what-is-cpu-scheduling-in-an-operating-system-0ec7256375e3

Goyal, S. (2024, February 16). CPU Scheduling In Operating Systems | All Concepts Explained. Unstop.com; Unstop. https://unstop.com/blog/cpu-scheduling-in-operating-system

GeeksforGeeks. (2020, June 16). CPU Scheduling Criteria. GeeksforGeeks. https://www.geeksforgeeks.org/operating-systems/cpu-scheduling-criteria/

Waseem, M. (2025, November 30). Exploring the Core Mechanics of CPU Scheduling Algorithms. Medium. https://medium.com/@mw0067785/exploring-the-core-mechanics-of-cpu-scheduling-algorithms-c1bd8bbf0f3d

GeeksforGeeks. (2020, March 18). FCFS First Come First Serve CPU Scheduling. GeeksforGeeks. https://www.geeksforgeeks.org/dsa/first-come-first-serve-cpu-scheduling-non-preemptive/

Team, D. (2021, August 28). FCFS Scheduling in OS - DataFlair. DataFlair. https://data-flair.training/blogs/fcfs-scheduling-in-os/

TheKnowledgeAcademy. (2024). First Come First Serve (FCFS) Scheduling Algorithm. Theknowledgeacademy.com. https://www.theknowledgeacademy.com/blog/first-come-first-serve/

DIRA. (2025, June 15). CPU Scheduling: Shortest Job First (SJF), Priority Scheduling, and Round Robin (RR). Medium. https://medium.com/@drajput_14416/cpu-scheduling-shortest-job-first-sjf-priority-scheduling-and-round-robin-rr-c42de2469950

www.naukri.com. (2024). Code 360 by Coding Ninjas. Naukri.com. https://www.naukri.com/code360/library/srtf-scheduling

GeeksforGeeks. (2019, November 7). *Introduction of Shortest Remaining Time First (SRTF) algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/operating-systems/introduction-of-shortest-remaining-time-first-srtf-algorithm/

Round Robin Scheduling Algorithm in Operating Systems. (2025). Tutorialspoint.com. https://www.tutorialspoint.com/operating_system/os_round_robin_scheduling_algorithm.htm

Priority Scheduling Algorithm in Operating System. (2021, September 8). DataFlair. https://data-flair.training/blogs/priority-scheduling-algorithm-in-operating-system/

Kaushik, H. (2025, July 6). *Priority-Based Round Robin Scheduling*. Medium. https://medium.com/@hardik.kaushik2003/priority-based-round-robin-scheduling-d1a2a0abc3f0

González-Rodríguez, M., Otero-Cerdeira, L., Encarnación González-Rufino, & Francisco Javier Rodríguez-Martínez. (2024). Study and Evaluation of CPU Scheduling Algorithms. *Heliyon*, e29959–e29959. https://doi.org/10.1016/j.heliyon.2024.e29959